

Universidad Mayor Real y Pontificia de San Francisco

Xavier de Chuquisaca

Facultad de Ciencias y Tecnología

Ingeniería en Ciencias de la Computación

SIS306 - Examen Final

SIS306 - Bases de Datos III

Docente: Ing. Edgar T. Espinoza Rodríguez

Estudiantes:

Ávila Serrano Christian Ángel

Peñaranda Villarroel Hernán Isaac

Limachi Villarroel Alan Nicolás

Fecha de entrega: Martes, 23 de junio de 2026

Diseño, transformación e implementación de una base de datos en

Oracle 19c para la gestión odontológica

## **Introducción**

El presente informe desarrolla la documentación técnica de una base de datos para la gestión de procesos odontológicos. El dominio considerado incluye registro de pacientes, asignación de carpetas clínicas, programación de citas, atención mediante odontogramas, tratamientos, planes de pago, registro de pagos, compras de suministros e inventario.

El trabajo toma como referencia documental una tesis previa sobre el desarrollo de un sistema de información web para el consultorio odontológico LALYSDENT. En dicha tesis se documenta un sistema web implementado con Laravel, MySQL y arquitectura MVC. A diferencia de esa investigación, el presente informe no se enfoca en el frontend ni en la aplicación web, sino en el diseño, transformación, implementación y validación de la base de datos.

La propuesta fue desarrollada en etapas. Primero, se definieron los requisitos y supuestos del dominio. Luego, se elaboró un modelo conceptual en notación Chen usando diagrams.net. Posteriormente, se realizó la transformación al modelo relacional y se construyó un modelo lógico Crow's Foot en ER/Studio 2003.

Finalmente, se implementó el diseño físico en Oracle Database 19c, ejecutado en Docker y administrado mediante DBeaver.

El documento incluye también un diccionario de datos completo, validaciones ejecutadas sobre Oracle 19c y una comparación técnica entre el enfoque MySQL/Laravel documentado en la tesis base y la transformación realizada hacia Oracle 19c.

## **Objetivos**

### **Objetivo general**

Diseñar, transformar, implementar y validar una base de datos relacional en Oracle Database 19c para la gestión de procesos odontológicos, tomando como referencia documental la tesis del sistema web LALYSDENT desarrollado con Laravel y MySQL.

**Objetivos específicos**

Identificar las entidades, atributos, relaciones, cardinalidades y restricciones del dominio odontológico documentado en la tesis base.

Elaborar el modelo conceptual en notación Chen usando diagrams.net.

Transformar el modelo conceptual al modelo relacional, definiendo tablas, claves primarias, claves foráneas, restricciones de dominio y relaciones opcionales u obligatorias.

Construir el modelo lógico en notación Crow's Foot mediante ER/Studio 2003.

Implementar el diseño físico en Oracle Database 19c, usando SQL y PL/SQL.

Crear un diccionario de datos completo y contrastarlo con el catálogo técnico de Oracle.

Comparar el enfoque MySQL/Laravel descrito en la tesis base con la transformación realizada hacia Oracle 19c.

Validar la base de datos mediante datos de prueba, consultas de control, triggers y procedimiento almacenado.

## Herramientas utilizadas

**Tabla 1**

*Herramientas utilizadas en el proyecto*

| Herramienta            | Uso en el proyecto                                                            |
|------------------------|-------------------------------------------------------------------------------|
| diagrams.net / Draw.io | Elaboración del modelo conceptual en notación Chen.                           |
| ER/Studio 2003         | Elaboración del modelo lógico relacional en notación Crow's Foot.             |
| Oracle Database 19c    | Motor de base de datos usado para la implementación física.                   |
| Docker                 | Ejecución de Oracle 19c en contenedor.                                        |
| DBeaver                | Ejecución de scripts SQL/PLSQL y validación de objetos.                       |
| VSCodium               | Edición del proyecto, scripts y documento final.                              |
| Konsole con zsh        | Ejecución de comandos, Git, compilación LaTeX y organización del repositorio. |
| LuaLaTeX + Biber       | Compilación del informe final con formato APA 7.                              |
| Git                    | Control de versiones del proyecto.                                            |

## Análisis de requisitos

### Alcance funcional tomado de la tesis base

La tesis base desarrolla un sistema de información web para administrar procesos de registro, atención, inventario y finanzas del consultorio odontológico LALYSIDENT. Dicho sistema incluye módulos de registro de pacientes, gestión de citas, historias clínicas con odontograma, administración de tratamientos, control de suministros e inventario odontológico, pagos de pacientes y reportes consolidados (Calvo Arteaga & Cobos Vargas, 2025a, 2025b).

En este examen final no se documenta el frontend ni la arquitectura completa de la aplicación web. El alcance se limita a la base de datos: diseño conceptual, transformación relacional, modelo lógico, implementación física, diccionario de

datos, validación y comparación técnica.

### **Entidades principales**

A partir del dominio se identificaron las siguientes entidades: asistente, folder, doctor, tratamiento, proveedor, suministro, paciente, cita, odontograma, detalle de odontograma, plan de pago, pago, compra, detalle de compra e inventario.

### **Procesos cubiertos**

La base de datos cubre cuatro procesos principales:

- Registro y citas: pacientes, carpetas clínicas, doctores, asistentes y citas.
- Atención odontológica: odontogramas, detalle por pieza dental y tratamientos.
- Finanzas y pagos: planes de pago y pagos registrados.
- Compras e inventario: proveedores, suministros, compras, detalle de compra y stock.

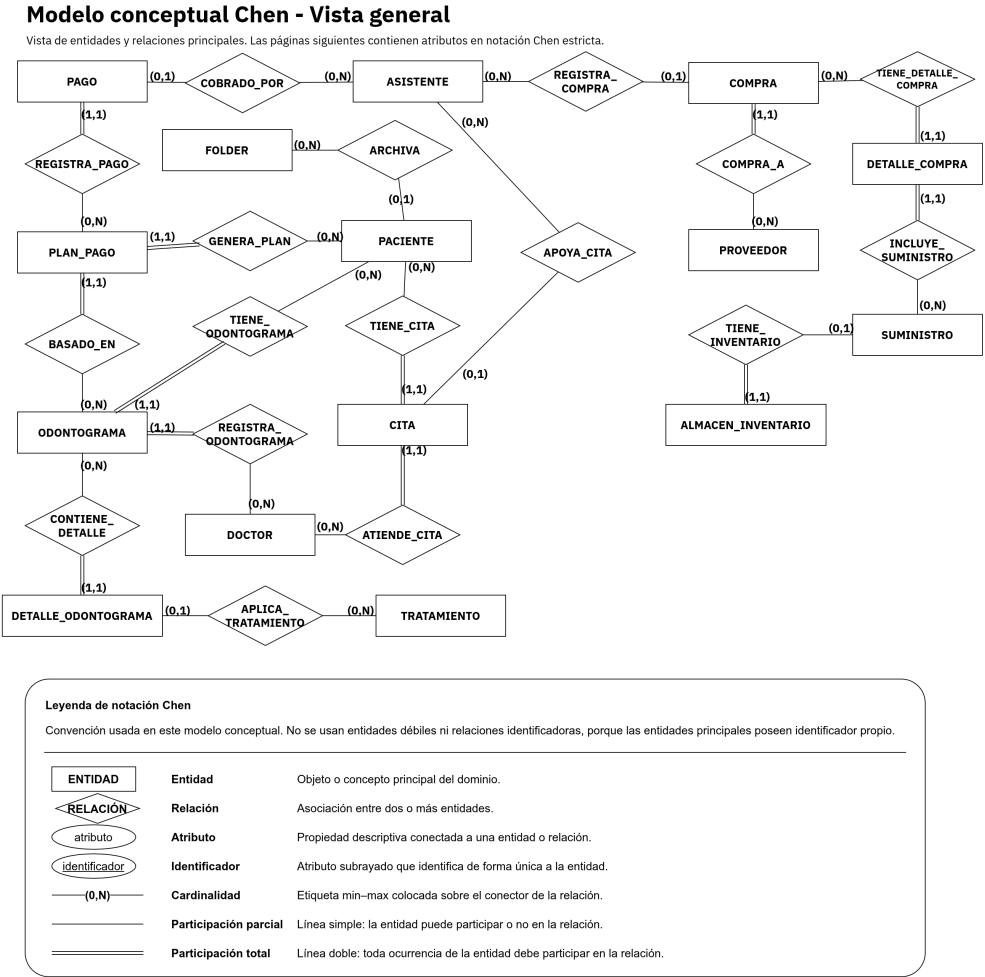
### **Supuestos de transformación**

No se dispuso de un DDL MySQL original completo de la tesis. Por ello, la comparación y transformación se realizaron a partir de la evidencia documental de la tesis y de los módulos funcionales descritos. La transformación no afirma una equivalencia sentencia por sentencia con un script MySQL original, sino una conversión técnica documentada del modelo de datos hacia Oracle 19c.

### **Modelo conceptual en notación Chen**

El modelo conceptual fue elaborado en Draw.io usando notación Chen estricta. En los diagramas se usaron rectángulos para entidades, rombos para relaciones, elipses para atributos, atributos subrayados para identificadores, líneas dobles para participación total y cardinalidad mínima-máxima junto a los conectores.

No se modelaron entidades débiles, porque las entidades principales poseen identificador propio. Para mejorar la legibilidad, el modelo se dividió en una vista general y cuatro vistas por módulo.



**Figura 1**

*Modelo conceptual Chen: vista general.*

Modelo conceptual Chen - Registro y citas

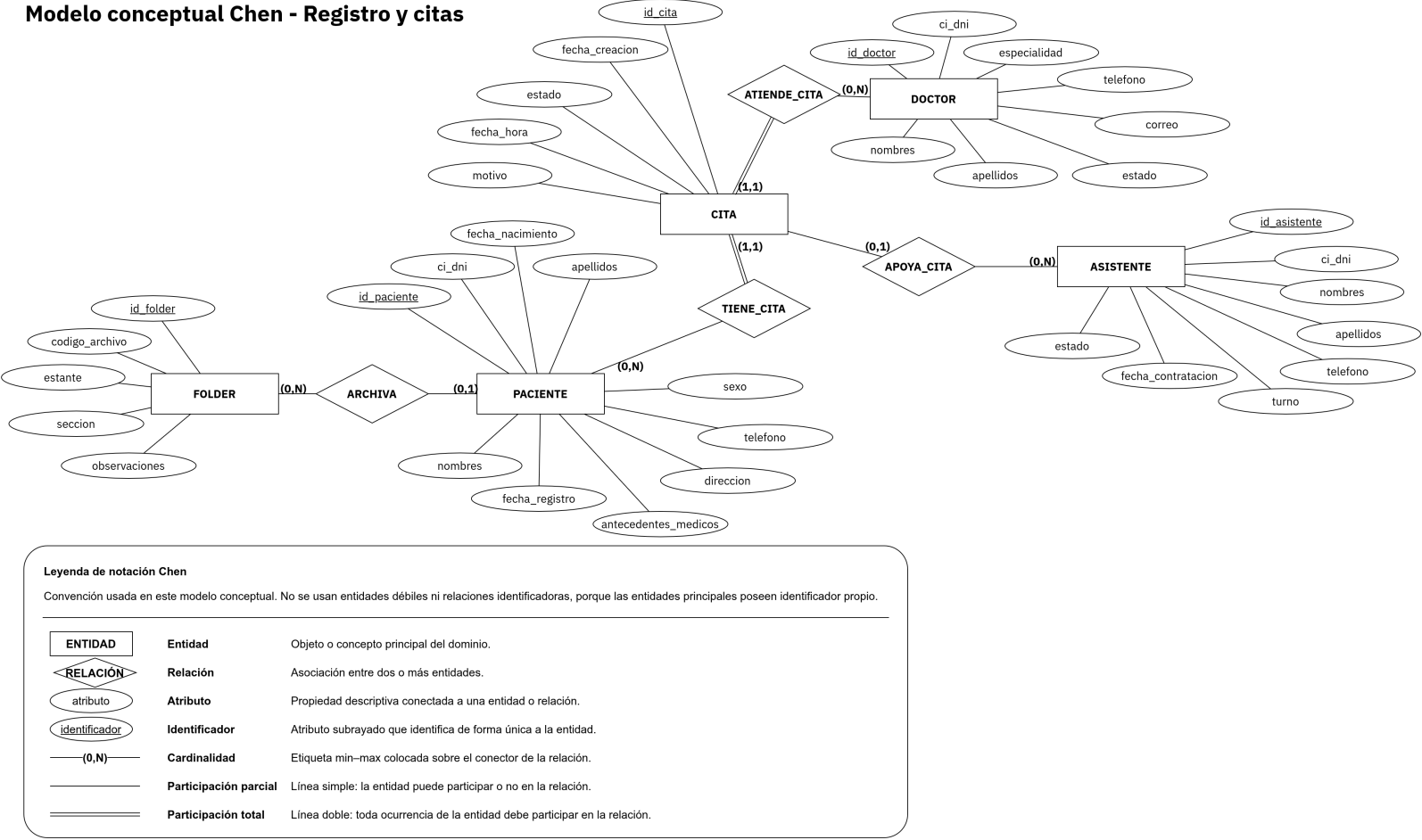
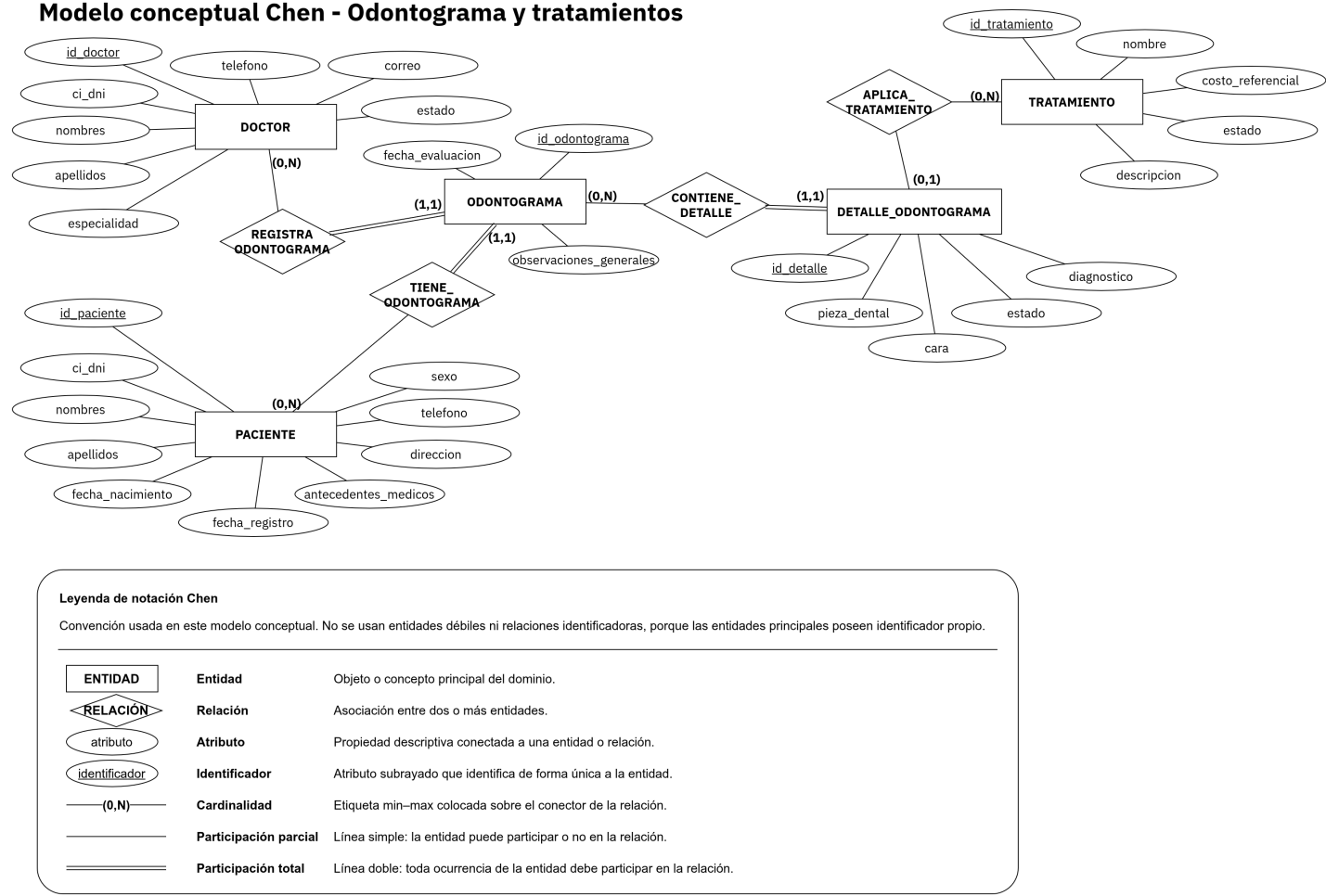


Figura 2

Modelo conceptual Chen: registro y citas.



**Figura 3**

*Modelo conceptual Chen: odontograma y tratamientos.*

Modelo conceptual Chen - Finanzas y pagos

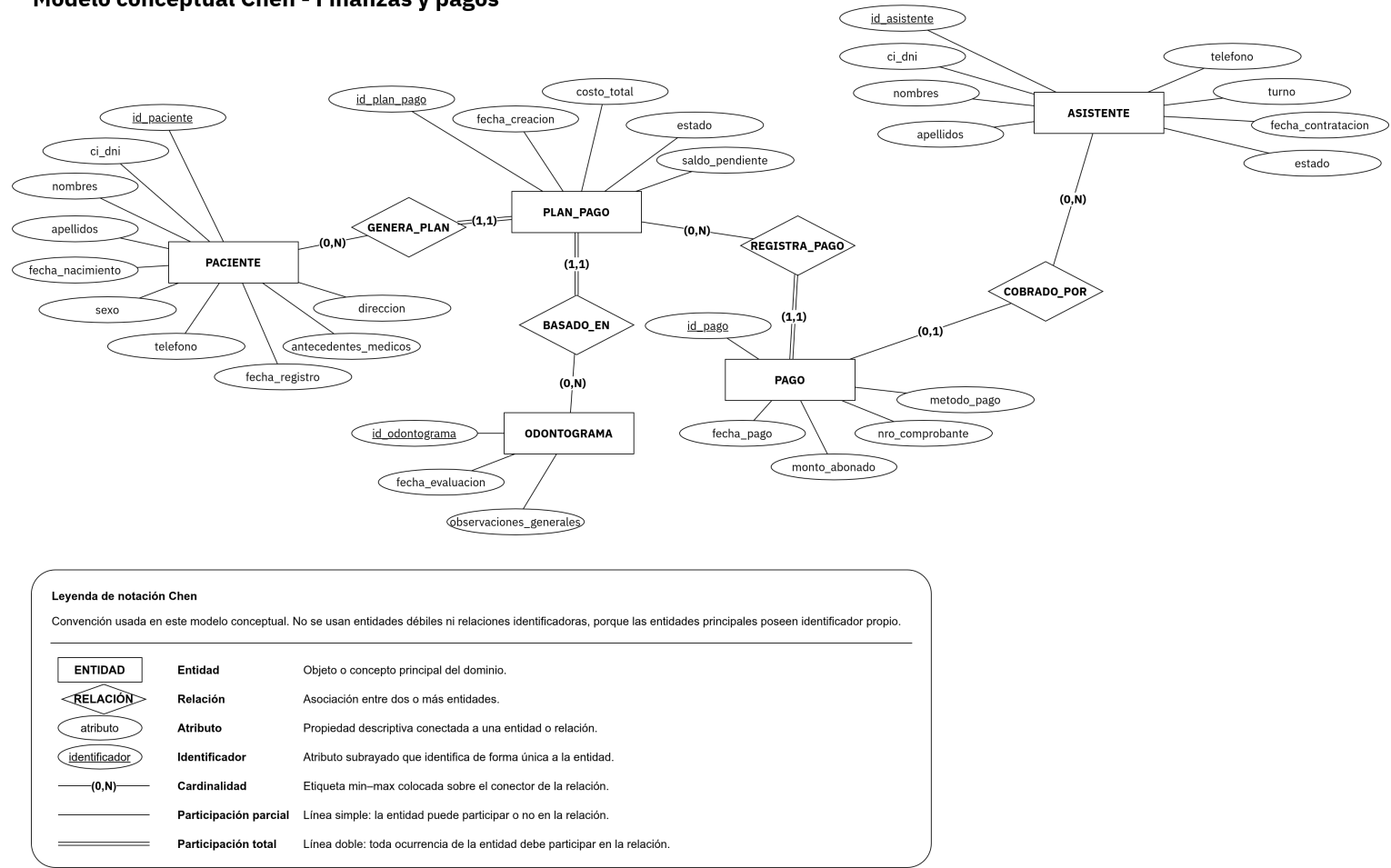


Figura 4

Modelo conceptual Chen: finanzas y pagos.

Modelo conceptual Chen - Compras e inventario

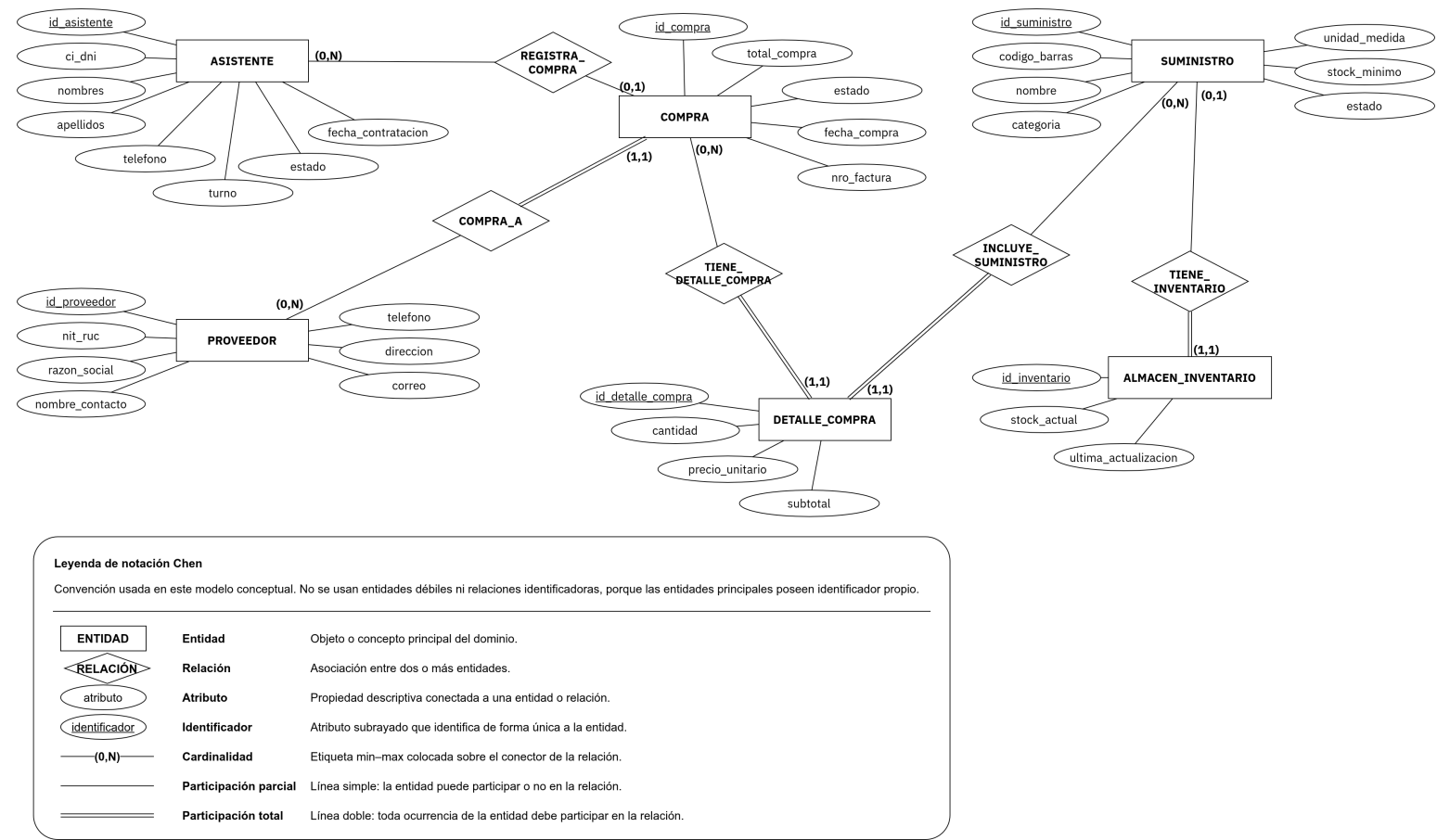


Figura 5

Modelo conceptual Chen: compras e inventario.

### **Transformación al modelo relacional**

La transformación del modelo conceptual al modelo relacional siguió reglas estándar de diseño de bases de datos. Cada entidad fuerte se transformó en una tabla, los identificadores se transformaron en claves primarias y las relaciones uno a muchos se materializaron mediante claves foráneas en el lado dependiente.

Las relaciones opcionales se representaron mediante columnas que aceptan valores nulos, mientras que las relaciones obligatorias se implementaron con claves foráneas NOT NULL. Las reglas de dominio simples se implementaron mediante restricciones CHECK, y las reglas operacionales se implementaron mediante triggers y procedimientos almacenados en Oracle 19c.

El resultado de la transformación produjo quince tablas relacionales, organizadas en los bloques de registro, atención clínica, finanzas e inventario.

### **Modelo lógico Crow's Foot**

El modelo lógico en Crow's Foot fue generado en ER/Studio 2003. En este modelo se visualizan las tablas, claves primarias, claves foráneas, relaciones, cardinalidades y obligatoriedad.

Para la importación se preparó un DDL simplificado compatible con Oracle 11g, debido a que dicha versión era la plataforma Oracle más reciente disponible en ER/Studio 2003. Este DDL no reemplaza al diseño físico Oracle 19c, sino que facilita la representación lógica de tablas y relaciones.



El modelo contiene las quince tablas del diseño relacional. Se observan las relaciones entre pacientes, citas, odontogramas, pagos, compras e inventario. Las claves foráneas importadas permiten visualizar la cardinalidad entre tablas padre e hija.

### **Diseño físico en Oracle 19c**

La implementación física se realizó en Oracle Database 19c dentro del schema FARMACIAS\_BD3. En Oracle, el schema está asociado al usuario propietario de los objetos, por lo que se creó un usuario específico para contener las tablas, vistas, triggers y procedimientos del proyecto.

**Compatibilidad de ejecución de los scripts..** Aunque las pruebas y evidencias del proyecto se ejecutaron principalmente en DBeaver, los scripts se mantuvieron con sintaxis estándar de Oracle 19c. En particular, los bloques PL/SQL, triggers y procedimientos conservan la barra diagonal / como delimitador de ejecución al finalizar el bloque. Por esta razón, el código puede ejecutarse también en Oracle SQL Developer, SQL\*Plus y otros clientes o IDEs compatibles con Oracle. La variante específica para DBeaver se conserva únicamente como alternativa práctica cuando dicho cliente divide incorrectamente un bloque PL/SQL durante la ejecución.

## Scripts implementados

**Tabla 2**

*Scripts físicos Oracle 19c*

| Script                          | Propósito                                                       |
|---------------------------------|-----------------------------------------------------------------|
| 00_admin_crear_schema.sql       | Crea el usuario/schema y asigna permisos.                       |
| 01_limpieza_esquema.sql         | Limpia objetos previos para permitir reejecución.               |
| 02_tablas.sql                   | Crea tablas, claves primarias, claves foráneas y restricciones. |
| 03_vistas.sql                   | Crea la vista de historial clínico.                             |
| 04_triggers.sql                 | Crea triggers de inventario y pagos.                            |
| 05_procedimientos.sql           | Define el procedimiento almacenado formal.                      |
| 05_procedimientos_dbeaver.r.sql | Variante compatible con DBeaver.                                |
| 06_datos_prueba.sql             | Carga datos de prueba.                                          |

## Objetos programables

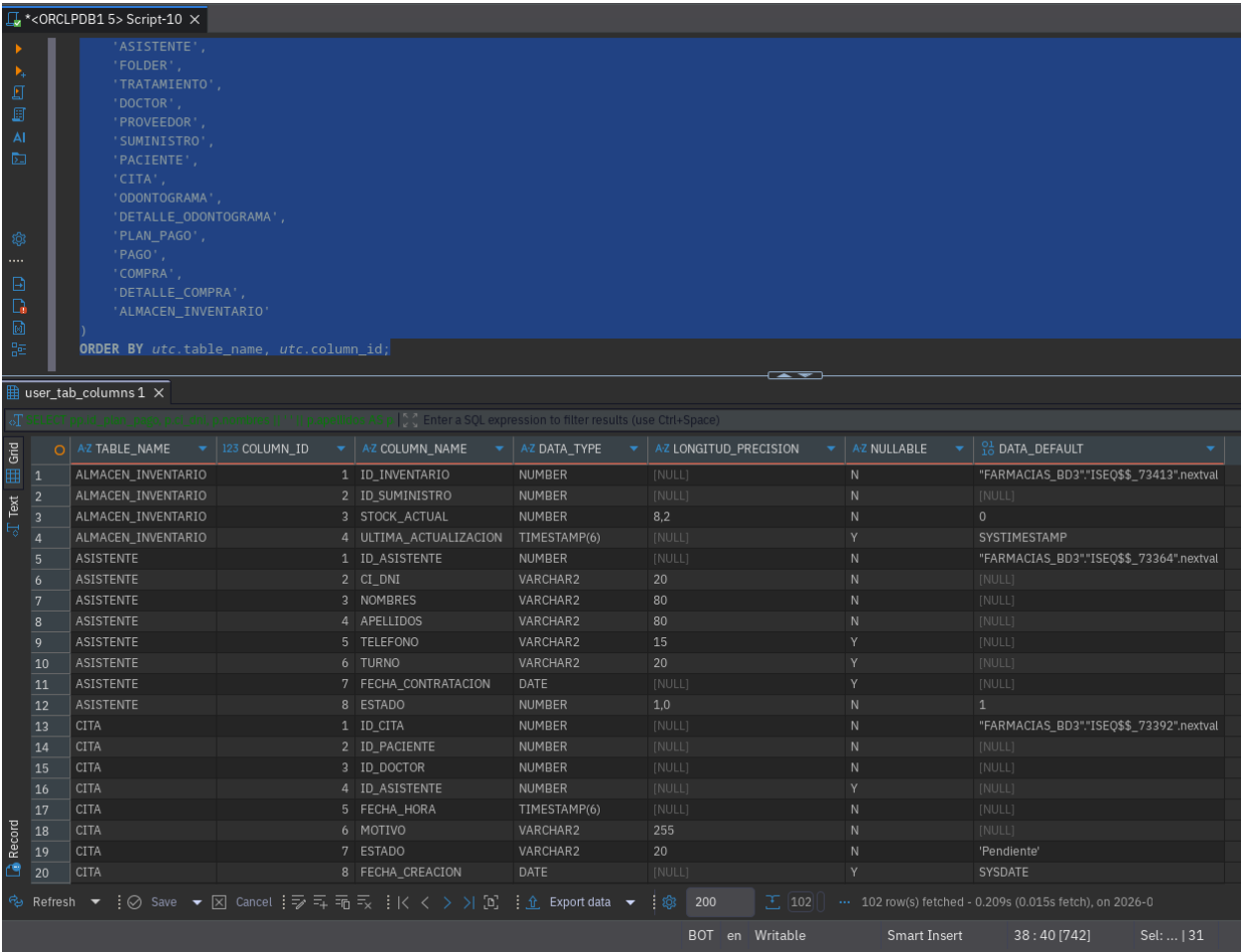
Se implementó la vista VW\_HISTORIAL\_CLINICO, el trigger TRG\_ACTUALIZAR\_STOCK\_COMPRA, el trigger TRG\_AUDITAR\_PLAN\_PAGO y el procedimiento almacenado SP\_REGISTRAR\_PAGO. Estos objetos permiten centralizar reglas de consulta, inventario y pagos directamente en el motor de base de datos.

## Diccionario de datos

El diccionario de datos completo se encuentra en el archivo 08\_diccionario\_datos/01\_diccionario\_datos.md. Este diccionario documenta cada tabla, campo, tipo de dato, nulabilidad, clave y descripción funcional.

Para evidenciar que el diccionario corresponde al esquema real, se ejecutó una

consulta técnica sobre el catálogo de Oracle mediante USER\_TAB\_COLUMNS, USER\_CONSTRAINTS y USER\_CONS\_COLUMNS. Esta validación permitió contrastar columnas, tipos de datos, restricciones y relaciones de claves foráneas.



**Figura 7**  
*Extracción técnica de columnas desde Oracle.*

SQL Developer Script window showing the following query:

```
-- 4. Relaciones de claves foráneas
SELECT
  child.table_name AS tabla_hija,
  child_cols.column_name AS columna_hija,
  child.constraint_name AS fk_name,
  parent.table_name AS tabla_padre,
  parent_cols.column_name AS columna_padre,
  child.delete_rule
FROM user_constraints child
JOIN user_cons_columns child_cols
  ON child.constraint_name = child_cols.constraint_name
JOIN user_constraints parent
  ON child.r_constraint_name = parent.constraint_name
JOIN user_cons_columns parent_cols
  ON parent.constraint_name = parent_cols.constraint_name
  AND child_cols.position = parent_cols.position
WHERE child.constraint_type = 'R'
ORDER BY child.table_name, child.constraint_name, child_cols.position;
```

The results are displayed in a table with the following columns: child table name, child column name, child constraint name, parent table name, parent column name, and delete rule. The table contains 17 rows of data.

| child | AZ TABLA_HIJA       | AZ COLUMNA_HIJA | AZ FK_NAME               | AZ TABLA_PADRE | AZ COLUMNA_PADRE | AZ DELETE_RULE |
|-------|---------------------|-----------------|--------------------------|----------------|------------------|----------------|
| 1     | ALMACEN_INVENTARIO  | ID_SUMINISTRO   | FK_INVENTARIO_SUMINISTRO | SUMINISTRO     | ID_SUMINISTRO    | NO ACTION      |
| 2     | CITA                | ID_ASISTENTE    | FK_CITA_ASISTENTE        | ASISTENTE      | ID_ASISTENTE     | NO ACTION      |
| 3     | CITA                | ID_DOCTOR       | FK_CITA_DOCTOR           | DOCTOR         | ID_DOCTOR        | NO ACTION      |
| 4     | CITA                | ID_PACIENTE     | FK_CITA_PACIENTE         | PACIENTE       | ID_PACIENTE      | NO ACTION      |
| 5     | COMPRA              | ID_ASISTENTE    | FK_COMPRA_ASISTENTE      | ASISTENTE      | ID_ASISTENTE     | NO ACTION      |
| 6     | COMPRA              | ID_PROVEEDOR    | FK_COMPRA_PROVEEDOR      | PROVEEDOR      | ID_PROVEEDOR     | NO ACTION      |
| 7     | DETALLE_COMPRA      | ID_COMPRA       | FK_DETCOMPRA_COMPRA      | COMPRA         | ID_COMPRA        | CASCADE        |
| 8     | DETALLE_COMPRA      | ID_SUMINISTRO   | FK_DETCOMPRA_SUMINISTRO  | SUMINISTRO     | ID_SUMINISTRO    | NO ACTION      |
| 9     | DETALLE_ODONTOGRAMA | ID_ODONTOGRAMA  | FK_DET_ODONTOGRAMA       | ODONTOGRAMA    | ID_ODONTOGRAMA   | CASCADE        |
| 10    | DETALLE_ODONTOGRAMA | ID_TRATAMIENTO  | FK_DET_TRATAMIENTO       | TRATAMIENTO    | ID_TRATAMIENTO   | NO ACTION      |
| 11    | ODONTOGRAMA         | ID_DOCTOR       | FK_ODONTO_DOCTOR         | DOCTOR         | ID_DOCTOR        | NO ACTION      |
| 12    | ODONTOGRAMA         | ID_PACIENTE     | FK_ODONTO_PACIENTE       | PACIENTE       | ID_PACIENTE      | NO ACTION      |
| 13    | PACIENTE            | ID_FOLDER       | FK_PACIENTE_FOLDER       | FOLDER         | ID_FOLDER        | SET NULL       |
| 14    | PAGO                | ID_ASISTENTE    | FK_PAGO_ASISTENTE        | ASISTENTE      | ID_ASISTENTE     | NO ACTION      |
| 15    | PAGO                | ID_PLAN_PAGO    | FK_PAGO_PLAN             | PLAN_PAGO      | ID_PLAN_PAGO     | NO ACTION      |
| 16    | PLAN_PAGO           | ID_ODONTOGRAMA  | FK_PLAN_ODONTO           | ODONTOGRAMA    | ID_ODONTOGRAMA   | NO ACTION      |
| 17    | PLAN_PAGO           | ID_PACIENTE     | FK_PLAN_PACIENTE         | PACIENTE       | ID_PACIENTE      | NO ACTION      |

**Figura 8**

*Extracción técnica de relaciones desde Oracle.*

## Validaciones

Las validaciones se ejecutaron sobre Oracle Database 19c mediante DBeaver.

Primero se validó la creación del schema, luego la creación de tablas, restricciones, vista, triggers y procedimiento. Posteriormente, se cargaron datos de prueba y se verificó el comportamiento del inventario y del registro de pagos.

The screenshot shows the Oracle SQL Developer interface. The top pane displays a SQL query in a script editor:

```
SELECT table_name  
FROM user_tables  
ORDER BY table_name;
```

The bottom pane shows the results of the query in a table grid. The table has two columns: 'TABLE\_NAME' and an empty column. The results are sorted alphabetically by table name.

|    | TABLE_NAME          |  |
|----|---------------------|--|
| 1  | ALMACEN_INVENTARIO  |  |
| 2  | ASISTENTE           |  |
| 3  | CITA                |  |
| 4  | COMPRA              |  |
| 5  | DETALLE_COMPRA      |  |
| 6  | DETALLE_ODONTOGRAMA |  |
| 7  | DOCTOR              |  |
| 8  | FOLDER              |  |
| 9  | ODONTOGRAMA         |  |
| 10 | PACIENTE            |  |
| 11 | PAGO                |  |
| 12 | PLAN_PAGO           |  |
| 13 | PROVEEDOR           |  |
| 14 | SUMINISTRO          |  |
| 15 | TRATAMIENTO         |  |

The bottom pane also includes a toolbar with options like Refresh, Save, Cancel, and Export data.

**Figura 9**

*Tablas creadas en el schema FARMACIAS\_BD3.*

The screenshot displays the Oracle SQL Developer interface. The top pane shows a SQL script named 'Script-10' with the following query:

```
SELECT object_name, object_type, status
FROM user_objects
WHERE object_type IN ('TRIGGER', 'PROCEDURE', 'VIEW')
ORDER BY object_type, object_name;
```

The bottom pane shows the results of the query in a table grid. The table has three columns: 'AZ OBJECT\_NAME', 'AZ OBJECT\_TYPE', and 'AZ STATUS'. The results are as follows:

|   | AZ OBJECT_NAME              | AZ OBJECT_TYPE | AZ STATUS |
|---|-----------------------------|----------------|-----------|
| 1 | TRG_ACTUALIZAR_STOCK_COMPRA | TRIGGER        | VALID     |
| 2 | TRG_AUDITAR_PLAN_PAGO       | TRIGGER        | VALID     |
| 3 | VW_HISTORIAL_CLINICO        | VIEW           | VALID     |

The interface also includes a sidebar on the left with icons for various database operations and a bottom toolbar with buttons for Refresh, Save, Cancel, and Export data.

**Figura 10**

*Vista y triggers validados en Oracle.*

The screenshot displays the Oracle SQL Developer environment. The top pane, titled '\*<ORCLPDB1 5> Script-10 X', contains the following SQL query:

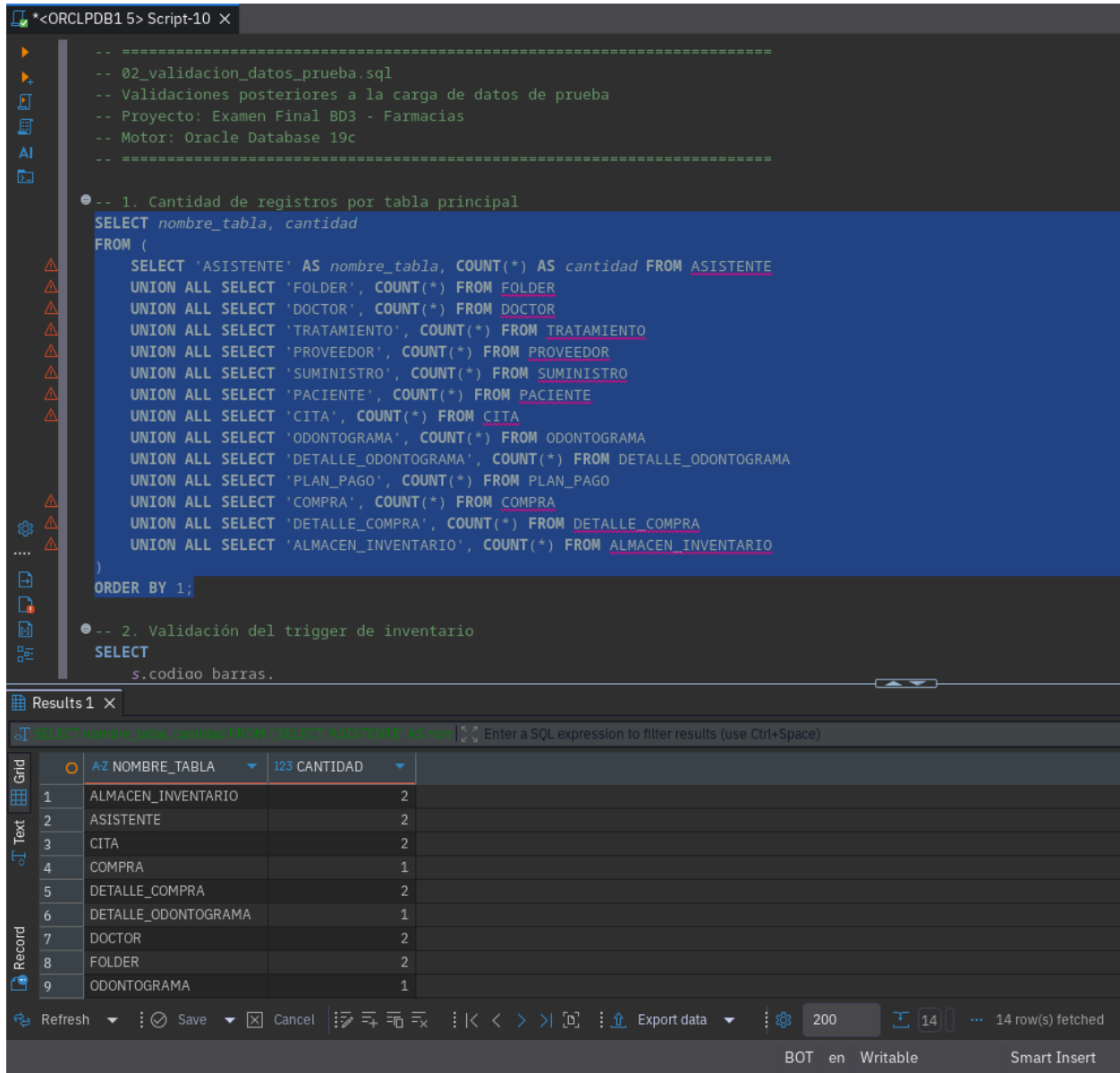
```
SELECT object_name, object_type, status
FROM user_objects
WHERE object_name = 'SP_REGISTRAR_PAGO';
```

The bottom pane, titled 'user\_objects 1 X', shows the results of the query in a table grid. The table has three columns: 'AZ OBJECT\_NAME', 'AZ OBJECT\_TYPE', and 'AZ STATUS'. The first row contains the data: 'SP\_REGISTRAR\_PAGO', 'PROCEDURE', and 'VALID'. The interface includes a toolbar with icons for settings, file operations, and a bottom status bar with buttons for 'Refresh', 'Save', 'Cancel', and 'Export data', along with a page size of 200 and a page number of 1.

|   | AZ OBJECT_NAME    | AZ OBJECT_TYPE | AZ STATUS |
|---|-------------------|----------------|-----------|
| 1 | SP_REGISTRAR_PAGO | PROCEDURE      | VALID     |
|   |                   |                |           |
|   |                   |                |           |
|   |                   |                |           |
|   |                   |                |           |
|   |                   |                |           |
|   |                   |                |           |
|   |                   |                |           |
|   |                   |                |           |
|   |                   |                |           |

**Figura 11**

*Procedimiento almacenado validado en Oracle.*



The screenshot shows the Oracle SQL Developer interface. The top pane displays a script named 'Script-10' with the following content:

```
-- *****
-- 02_validacion_datos_prueba.sql
-- Validaciones posteriores a la carga de datos de prueba
-- Proyecto: Examen Final BD3 - Farmacias
-- Motor: Oracle Database 19c
-- *****

-- 1. Cantidad de registros por tabla principal
SELECT nombre_tabla, cantidad
FROM (
  SELECT 'ASISTENTE' AS nombre_tabla, COUNT(*) AS cantidad FROM ASISTENTE
  UNION ALL SELECT 'FOLDER', COUNT(*) FROM FOLDER
  UNION ALL SELECT 'DOCTOR', COUNT(*) FROM DOCTOR
  UNION ALL SELECT 'TRATAMIENTO', COUNT(*) FROM TRATAMIENTO
  UNION ALL SELECT 'PROVEEDOR', COUNT(*) FROM PROVEEDOR
  UNION ALL SELECT 'SUMINISTRO', COUNT(*) FROM SUMINISTRO
  UNION ALL SELECT 'PACIENTE', COUNT(*) FROM PACIENTE
  UNION ALL SELECT 'CITA', COUNT(*) FROM CITA
  UNION ALL SELECT 'ODONTOGRAMA', COUNT(*) FROM ODONTOGRAMA
  UNION ALL SELECT 'DETALLE_ODONTOGRAMA', COUNT(*) FROM DETALLE_ODONTOGRAMA
  UNION ALL SELECT 'PLAN_PAGO', COUNT(*) FROM PLAN_PAGO
  UNION ALL SELECT 'COMPRA', COUNT(*) FROM COMPRA
  UNION ALL SELECT 'DETALLE_COMPRA', COUNT(*) FROM DETALLE_COMPRA
  UNION ALL SELECT 'ALMACEN_INVENTARIO', COUNT(*) FROM ALMACEN_INVENTARIO
)
ORDER BY 1;

-- 2. Validación del trigger de inventario
SELECT
  s.codigo barras.
```

The bottom pane shows the 'Results 1' window with a grid view of the query results:

|   | AZ NOMBRE_TABLA     | 123 CANTIDAD |
|---|---------------------|--------------|
| 1 | ALMACEN_INVENTARIO  | 2            |
| 2 | ASISTENTE           | 2            |
| 3 | CITA                | 2            |
| 4 | COMPRA              | 1            |
| 5 | DETALLE_COMPRA      | 2            |
| 6 | DETALLE_ODONTOGRAMA | 1            |
| 7 | DOCTOR              | 2            |
| 8 | FOLDER              | 2            |
| 9 | ODONTOGRAMA         | 1            |

The status bar at the bottom indicates '14 row(s) fetched' and 'BOT en Writable Smart Insert'.

**Figura 12**

*Validación de datos de prueba cargados.*

The screenshot shows the Oracle SQL Developer interface. The top pane is a script editor with the following SQL code:

```

p_metodo_pago => 'Efectivo',
p_comprobante => 'REC-001'
);
END;

-- Pagos registrados
SELECT
  pg.id_pago,
  pg.id_plan_pago,
  pg.id_asistente,
  pg.monto_abonado,
  pg.metodo_pago,
  pg.nro_comprobante,
  pg.fecha_pago
FROM PAGO pg
ORDER BY pg.id_pago;

-- Saldo después del pago
SELECT
  pp.id_plan_pago,
  p.ci_dni,
  p.nombres || ' ' || p.apellidos AS paciente,
  pp.costo_total,
  pp.saldo_pendiente,
  pp.estado
FROM PLAN_PAGO pp
JOIN PACIENTE p
  ON pp.id_paciente = p.id_paciente
WHERE p.ci_dni = 'P001';

```

The bottom pane shows the query result grid for the second query. The grid has the following columns: ID\_PLAN\_PAGO, CI\_DNI, PACIENTE, COSTO\_TOTAL, SALDO\_PENDIENTE, and ESTADO. The first row of data is highlighted.

| ID_PLAN_PAGO | CI_DNI | PACIENTE           | COSTO_TOTAL | SALDO_PENDIENTE | ESTADO  |
|--------------|--------|--------------------|-------------|-----------------|---------|
| 1            | P001   | Daniel Choque Lima | 180         | 100             | Vigente |

The status bar at the bottom indicates "1 row(s) fetched - 0.002s, o" and "BOT en Writable Smart Insert".

**Figura 13**

*Prueba funcional del procedimiento de pago.*

## Conversión del enfoque MySQL/Laravel de la tesis a Oracle 19c

### Punto de partida documental

La tesis base plantea un sistema web para el consultorio odontológico LALYSIDENT. El sistema fue desarrollado con Laravel 11, Jetstream, Livewire, Tailwind CSS, JQuery y MySQL, bajo arquitectura MVC. Su alcance funcional incluye registro de

pacientes, citas, historias clínicas, odontograma, tratamientos, inventario, pagos y reportes consolidados (Calvo Arteaga & Cobos Vargas, 2025a, 2025b).

La conversión realizada en este proyecto toma ese dominio como referencia, pero cambia el foco: en lugar de documentar la aplicación web, se documenta la base de datos como producto principal. Por ello, el resultado no es una migración automática de un dump MySQL, sino una transformación técnica del diseño de datos hacia Oracle Database 19c.

**Diferencia de enfoque**

En el enfoque Laravel/MySQL, MySQL actúa como repositorio relacional para una aplicación web cuya lógica se distribuye entre modelos, controladores, vistas, componentes Livewire y validaciones del framework. En la transformación a Oracle 19c, varias reglas críticas se expresan directamente dentro del motor mediante restricciones, triggers, vista y procedimiento almacenado.

**Tabla 3**

*Cambio de enfoque entre MySQL/Laravel y Oracle 19c*

| Aspecto             | MySQL/Laravel en la tesis                            | Oracle 19c en este proyecto                            |
|---------------------|------------------------------------------------------|--------------------------------------------------------|
| Propósito principal | Persistencia para un sistema web odontológico.       | Diseño e implementación formal de base de datos.       |
| Capa dominante      | Aplicación Laravel bajo MVC.                         | Motor Oracle con SQL y PL/SQL.                         |
| Validaciones        | Formularios, modelos, controladores y base de datos. | Restricciones, claves, triggers y procedimiento.       |
| Automatización      | Principalmente desde la lógica de aplicación.        | Triggers y procedimiento almacenado.                   |
| Evidencia           | Funcionalidad del sistema web.                       | Scripts, consultas, catálogo Oracle y datos de prueba. |

### Conversión de tipos de datos

La conversión física requirió adaptar tipos usados normalmente en MySQL o Laravel hacia tipos Oracle equivalentes. La tabla siguiente resume el criterio aplicado.

**Tabla 4**

*Conversión de tipos de datos hacia Oracle 19c*

| Necesidad                   | MySQL/Laravel                                     | Oracle 19c aplicado                   |
|-----------------------------|---------------------------------------------------|---------------------------------------|
| Clave primaria autogenerada | INTAUTO_INCREMENT o migraciones Laravel con id(). | NUMBERGENERATEDALWAYSASIDENTITY.      |
| Texto variable              | VARCHAR.                                          | VARCHAR2(n).                          |
| Texto fijo corto            | CHAR.                                             | CHAR(1).                              |
| Fecha simple                | DATE.                                             | DATE.                                 |
| Fecha y hora                | DATETIME o TIMESTAMP.                             | TIMESTAMP.                            |
| Monto monetario             | DECIMAL(p,s).                                     | NUMBER(p,s).                          |
| Estado lógico               | Booleano, tinyint o entero.                       | NUMBER(1) o VARCHAR2 con restricción. |
| Fecha actual                | NOW() o CURRENT_TIMESTAMP.                        | SYSDATE o SYSTIMESTAMP.               |

### Conversión de módulos funcionales a tablas Oracle

Los módulos funcionales descritos en la tesis se transformaron en tablas y relaciones explícitas. El módulo de pacientes se convirtió en la tabla PACIENTE; la gestión documental se representó mediante FOLDER; las citas mediante CITA; la atención clínica mediante ODONTOGRAMA y DETALLE\_ODONTOGRAMA; los tratamientos mediante TRATAMIENTO; los pagos mediante PLAN\_PAGO y PAGO; y el inventario mediante SUMINISTRO, COMPRA, DETALLE\_COMPRA y

ALMACEN\_INVENTARIO.

**Tabla 5***Transformación de módulos de la tesis a objetos Oracle*

| <b>Módulo documentado</b>     | <b>Interpretación de datos</b>                   | <b>Objeto Oracle resultante</b>                         |
|-------------------------------|--------------------------------------------------|---------------------------------------------------------|
| Registro de pacientes         | Datos personales, contacto y antecedentes.       | PACIENTE.                                               |
| Gestión de historias clínicas | Organización documental y clínica.               | FOLDER, ODONTOGRAMA.                                    |
| Citas odontológicas           | Programación entre paciente, doctor y asistente. | CITA.                                                   |
| Odontograma                   | Evaluación dental por pieza, cara y diagnóstico. | ODONTOGRAMA, DETALLE_ODONTOGRAMA.                       |
| Tratamientos                  | Catálogo clínico de tratamientos.                | TRATAMIENTO.                                            |
| Inventario odontológico       | Suministros, compras y stock.                    | SUMINISTRO, COMPRA, DETALLE_COMPRA, ALMACEN_INVENTARIO. |
| Pagos de pacientes            | Plan financiero y abonos realizados.             | PLAN_PAGO, PAGO.                                        |
| Reportes consolidados         | Consultas integradas de datos clínicos.          | VW_HISTORIAL_CLINICO.                                   |

**Conversión de reglas de integridad**

En Laravel, muchas reglas pueden implementarse como validaciones de formulario, reglas de modelo o lógica de controlador. En Oracle 19c se decidió trasladar reglas críticas al motor para impedir inconsistencias aunque los datos sean manipulados desde DBeaver u otro cliente SQL.

**Tabla 6***Conversión de reglas de negocio a restricciones Oracle*

| <b>Regla</b>                  | <b>Enfoque Laravel/MySQL</b>                  | <b>Implementación Oracle 19c</b>        |
|-------------------------------|-----------------------------------------------|-----------------------------------------|
| Documento único de paciente   | Validación de formulario y restricción única. | UNIQUE(PACIENTE.ci_dni).                |
| Doctor con correo único       | Validación en aplicación o índice único.      | UNIQUE(DOCTOR.correo).                  |
| Estado válido de cita         | Validación desde formulario o controlador.    | CHECK sobre CITA.estado.                |
| Pago positivo                 | Validación de entrada y regla de negocio.     | CHECK(monto_abonado > 0).               |
| Saldo no negativo             | Lógica de aplicación.                         | Trigger<br>TRG_AUDITAR_PLAN_PAGO.       |
| Compra actualiza inventario   | Servicio o controlador Laravel.               | Trigger<br>TRG_ACTUALIZAR_STOCK_COMPRA. |
| Registro de pago consistente  | Transacción en aplicación.                    | Procedimiento<br>SP_REGISTRAR_PAGO.     |
| Consulta de historial clínico | Consulta desde modelos/controladores.         | Vista<br>VW_HISTORIAL_CLINICO.          |

**Conversión de relaciones**

Las relaciones que en Laravel podrían expresarse mediante modelos Eloquent se materializaron como claves foráneas en Oracle. La decisión permite que la integridad referencial no dependa únicamente de la aplicación.

- CITA referencia a PACIENTE, DOCTOR y opcionalmente a ASISTENTE.
- ODONTOGRAMA referencia a PACIENTE y DOCTOR.

- DETALLE\_ODONTOGRAMA referencia a ODONTOGRAMA y opcionalmente a TRATAMIENTO.
- PLAN\_PAGO referencia a PACIENTE y ODONTOGRAMA.
- PAGO referencia a PLAN\_PAGO y opcionalmente a ASISTENTE.
- COMPRA referencia a PROVEEDOR y opcionalmente a ASISTENTE.
- DETALLE\_COMPRA referencia a COMPRA y SUMINISTRO.
- ALMACEN\_INVENTARIO referencia de forma única a SUMINISTRO.

### **Resultado de la transformación**

El resultado final de la conversión es un schema Oracle llamado FARMACIAS\_BD3, compuesto por quince tablas, una vista, dos triggers, un procedimiento almacenado, scripts de datos de prueba y scripts de validación. Esta implementación permite demostrar no solo la estructura de datos, sino también el comportamiento del modelo ante operaciones reales como compras y pagos.

### **Limitación de la comparación**

La comparación se basa en la tesis y en su sitio web de repositorio, no en un dump MySQL original. Por esa razón, las equivalencias se expresan a nivel de dominio, modelo de datos, tipos, reglas y objetos Oracle. No se presenta como una migración automática línea por línea.

### **Fuentes del trabajo**

En este informe, el término fuentes se refiere a los archivos, diagramas, scripts y evidencias generados durante el desarrollo del proyecto. Estos artefactos se encuentran organizados dentro del repositorio del examen final.

**Tabla 7***Fuentes y artefactos del trabajo*

| <b>Categoría</b>             | <b>Archivo o evidencia</b>                                      |
|------------------------------|-----------------------------------------------------------------|
| Requisitos                   | 00_requisitos/01_alcance_y_supuestos.md                         |
| Modelo Chen editable         | 01_conceptual_chen/modelo_conceptual_chen_bd3.drawio            |
| Modelo Chen exportado        | 01_conceptual_chen/modelo_chen_*.png                            |
| Transformación lógica        | 02_transformacion_logica/01_transformacion_modelo_relacional.md |
| Modelo Crow's Foot           | 03_crows_foot/modelo_logico_crows_foot_bd3.DM1                  |
| Modelo Crow's Foot exportado | 03_crows_foot/modelo_logico_crows_foot_bd3.jpg                  |
| DDL y PL/SQL Oracle          | 05_oracle_ddl/*.sql                                             |
| Evidencias                   | 06_evidencias/*.png                                             |
| Diccionario de datos         | 08_diccionario_datos/01_diccionario_datos.md                    |
| Comparación MySQL/Oracle     | 09_comparacion_mysql_oracle/*.md                                |
| Validaciones                 | 10_validaciones/*.sql                                           |

### Conclusiones

Se diseñó e implementó una base de datos relacional para procesos odontológicos, cubriendo registro de pacientes, citas, odontogramas, tratamientos, pagos, compras e inventario.

El modelo conceptual Chen permitió representar el dominio sin depender de un motor físico. La transformación relacional permitió derivar tablas, claves y relaciones. El modelo lógico Crow's Foot facilitó la visualización de la estructura relacional, mientras que Oracle Database 19c permitió implementar el diseño físico

con restricciones, vista, triggers y procedimiento almacenado.

Las validaciones realizadas demostraron que los objetos fueron creados correctamente y que las reglas implementadas funcionan. En particular, se verificó la actualización de inventario al registrar compras y la actualización de saldo al registrar pagos mediante el procedimiento SP\_REGISTRAR\_PAGO.

Finalmente, la comparación con el enfoque MySQL/Laravel descrito en la tesis base permitió evidenciar que Oracle 19c ofrece una implementación más centrada en reglas del motor, integridad relacional explícita y validación técnica mediante el catálogo de base de datos.

### Referencias

- Calvo Arteaga, C. A., & Cobos Vargas, I. J. (2025a). Desarrollo de un sistema de información web para la administración de los procesos de registro, atención, inventario y finanzas del consultorio odontológico LALYSDENT del distrito de Cusco [Tesis usada como referencia documental para el análisis del dominio y comparación MySQL/Laravel].
- Calvo Arteaga, C. A., & Cobos Vargas, I. J. (2025b). *Repositorio de tesis: Desarrollo de un sistema de información web para la administración de los procesos de registro, atención, inventario y finanzas del consultorio odontológico Lalysdent*. Consultado el 20 de junio de 2026, desde <https://calvocobos.github.io/>
- Chen, P. P.-S. (1976). The Entity-Relationship Model: Toward a Unified View of Data.
- DBeaver. (2024). DBeaver Documentation [Documentación oficial de DBeaver para ejecución de scripts SQL]. <https://dbeaver.com/docs/dbeaver/>
- JGraph Ltd. (2024). diagrams.net Documentation [Documentación oficial de diagrams.net/draw.io]. <https://www.drawio.com/doc/>
- Oracle. (2024). Oracle Database 19c Documentation [Documentación oficial consultada para características generales de Oracle Database 19c]. <https://docs.oracle.com/en/database/oracle/oracle-database/19/>

## Apéndice A

### Anexo: scripts SQL y PL/SQL

Este anexo incorpora los scripts usados para crear, implementar, cargar y validar la base de datos en Oracle 19c.

#### Creación de schema/usuario

El siguiente script corresponde al archivo administrativo 05\_oracle\_ddl/00\_admin\_crear\_schema.sql. Su propósito es crear el usuario/schema FARMACIAS\_BD3 y asignar los permisos necesarios para la implementación de tablas, vistas, triggers y procedimientos en Oracle 19c.

Listing 1: Creación de usuario/schema y permisos en Oracle 19c

```
-- =====
-- 00_admin_crear_schema.sql
-- Creación de schema/usuario para el proyecto
-- Motor: Oracle Database 19c
-- =====

SELECT SYS_CONTEXT('USERENV', 'CON_NAME') AS contenedor_actual
FROM dual;

CREATE USER farmacias_bd3
IDENTIFIED BY FarmaciasBD3_2026
DEFAULT TABLESPACE users
TEMPORARY TABLESPACE temp
QUOTA UNLIMITED ON users;

GRANT CREATE SESSION TO farmacias_bd3;
GRANT CREATE TABLE TO farmacias_bd3;
GRANT CREATE VIEW TO farmacias_bd3;
GRANT CREATE SEQUENCE TO farmacias_bd3;
GRANT CREATE TRIGGER TO farmacias_bd3;
```

```
GRANT CREATE PROCEDURE TO farmacias_bd3;
```

```
SELECT username,
        account_status,
        default_tablespace,
        temporary_tablespace
FROM dba_users
WHERE username = 'FARMACIAS_BD3';
```

## Limpieza de esquema

```
-- =====
-- 01_limpieza_esquema.sql
-- Limpieza idempotente de objetos del esquema
-- Proyecto: Examen Final BD3 - Farmacias
-- Motor: Oracle Database 19c
-- =====

BEGIN
  FOR cur_rec IN (SELECT table_name FROM user_tables WHERE table_name IN (
    'ALMACEN_INVENTARIO', 'DETALLE_COMPRA', 'COMPRA',
    'PAGO', 'PLAN_PAGO', 'DETALLE_ODONTOGRAMA', 'ODONTOGRAMA',
    'CITA', 'PACIENTE', 'SUMINISTRO', 'PROVEEDOR',
    'DOCTOR', 'TRATAMIENTO', 'FOLDER', 'ASISTENTE'))
  LOOP
    EXECUTE IMMEDIATE 'DROP TABLE ' || cur_rec.table_name || ' CASCADE
    CONSTRAINTS';
  END LOOP;
END;
/
```

## DDL físico Oracle 19c

```
-- =====
```

```
-- 02_tablas.sql
-- Creación de tablas, claves primarias, claves foráneas y restricciones
-- Proyecto: Examen Final BD3 - Farmacias
-- Motor: Oracle Database 19c
-- =====

CREATE TABLE ASISTENTE (
    id_asistente NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ci_dni VARCHAR2(20) UNIQUE NOT NULL,
    nombres VARCHAR2(80) NOT NULL,
    apellidos VARCHAR2(80) NOT NULL,
    telefono VARCHAR2(15),
    turno VARCHAR2(20),
    fecha_contratacion DATE,
    estado NUMBER(1) DEFAULT 1 NOT NULL
);

CREATE TABLE FOLDER (
    id_folder NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    codigo_archivo VARCHAR2(20) UNIQUE NOT NULL,
    estante VARCHAR2(20),
    seccion VARCHAR2(20),
    observaciones VARCHAR2(255)
);

CREATE TABLE TRATAMIENTO (
    id_tratamiento NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR2(100) NOT NULL,
    descripcion VARCHAR2(255),
    costo_referencial NUMBER(10, 2) NOT NULL,
    estado NUMBER(1) DEFAULT 1 NOT NULL
);
```

```
CREATE TABLE DOCTOR (  
    id_doctor NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    ci_dni VARCHAR2(20) UNIQUE NOT NULL,  
    nombres VARCHAR2(80) NOT NULL,  
    apellidos VARCHAR2(80) NOT NULL,  
    especialidad VARCHAR2(100),  
    telefono VARCHAR2(15),  
    correo VARCHAR2(100) UNIQUE,  
    estado NUMBER(1) DEFAULT 1 NOT NULL  
);  
  
CREATE TABLE PROVEEDOR (  
    id_proveedor NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    nit_ruc VARCHAR2(20) UNIQUE NOT NULL,  
    razon_social VARCHAR2(100) NOT NULL,  
    nombre_contacto VARCHAR2(80),  
    telefono VARCHAR2(15),  
    direccion VARCHAR2(255),  
    correo VARCHAR2(100)  
);  
  
CREATE TABLE SUMINISTRO (  
    id_suministro NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    codigo_barras VARCHAR2(50) UNIQUE,  
    nombre VARCHAR2(100) NOT NULL,  
    categoria VARCHAR2(50),  
    unidad_medida VARCHAR2(20),  
    stock_minimo NUMBER(5) DEFAULT 5 NOT NULL,  
    estado NUMBER(1) DEFAULT 1 NOT NULL  
);  
  
CREATE TABLE PACIENTE (  
    id_paciente NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
id_folder NUMBER,
ci_dni VARCHAR2(20) UNIQUE NOT NULL,
nombres VARCHAR2(80) NOT NULL,
apellidos VARCHAR2(80) NOT NULL,
fecha_nacimiento DATE NOT NULL,
sexo CHAR(1),
telefono VARCHAR2(15),
direccion VARCHAR2(255),
antecedentes_medicos VARCHAR2(500),
fecha_registro DATE DEFAULT SYSDATE,
CONSTRAINT fk_paciente_folder FOREIGN KEY (id_folder)
    REFERENCES FOLDER(id_folder) ON DELETE SET NULL
);

CREATE TABLE CITA (
    id_cita NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_paciente NUMBER NOT NULL,
    id_doctor NUMBER NOT NULL,
    id_asistente NUMBER,
    fecha_hora TIMESTAMP NOT NULL,
    motivo VARCHAR2(255) NOT NULL,
    estado VARCHAR2(20) DEFAULT 'Pendiente' NOT NULL,
    fecha_creacion DATE DEFAULT SYSDATE,
    CONSTRAINT fk_cita_paciente FOREIGN KEY (id_paciente)
        REFERENCES PACIENTE(id_paciente),
    CONSTRAINT fk_cita_doctor FOREIGN KEY (id_doctor)
        REFERENCES DOCTOR(id_doctor),
    CONSTRAINT fk_cita_asistente FOREIGN KEY (id_asistente)
        REFERENCES ASISTENTE(id_asistente),
    CONSTRAINT chk_estado_cita
        CHECK (estado IN ('Pendiente', 'Confirmada', 'Atendida', 'Cancelada',
            'Reprogramada'))
);
```

```
CREATE TABLE ODONTOGRAMA (  
    id_odontograma NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_paciente NUMBER NOT NULL,  
    id_doctor NUMBER NOT NULL,  
    fecha_evaluacion DATE DEFAULT SYSDATE NOT NULL,  
    observaciones_generales VARCHAR2(500),  
    CONSTRAINT fk_odonto_paciente FOREIGN KEY (id_paciente)  
        REFERENCES PACIENTE(id_paciente),  
    CONSTRAINT fk_odonto_doctor FOREIGN KEY (id_doctor)  
        REFERENCES DOCTOR(id_doctor)  
);
```

```
CREATE TABLE DETALLE_ODONTOGRAMA (  
    id_detalle NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_odontograma NUMBER NOT NULL,  
    id_tratamiento NUMBER,  
    pieza_dental VARCHAR2(10) NOT NULL,  
    cara VARCHAR2(20) NOT NULL,  
    diagnostico VARCHAR2(100) NOT NULL,  
    estado VARCHAR2(20) DEFAULT 'Por tratar',  
    CONSTRAINT fk_det_odontograma FOREIGN KEY (id_odontograma)  
        REFERENCES ODONTOGRAMA(id_odontograma) ON DELETE CASCADE,  
    CONSTRAINT fk_det_tratamiento FOREIGN KEY (id_tratamiento)  
        REFERENCES TRATAMIENTO(id_tratamiento)  
);
```

```
CREATE TABLE PLAN_PAGO (  
    id_plan_pago NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_paciente NUMBER NOT NULL,  
    id_odontograma NUMBER NOT NULL,  
    fecha_creacion DATE DEFAULT SYSDATE,  
    costo_total NUMBER(10, 2) NOT NULL,
```

```
    saldo_pendiente NUMBER(10, 2) NOT NULL,
    estado VARCHAR2(20) DEFAULT 'Vigente',
    CONSTRAINT fk_plan_paciente FOREIGN KEY (id_paciente)
        REFERENCES PACIENTE(id_paciente),
    CONSTRAINT fk_plan_odonto FOREIGN KEY (id_odontograma)
        REFERENCES ODONTOGRAMA(id_odontograma)
);

CREATE TABLE PAGO (
    id_pago NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_plan_pago NUMBER NOT NULL,
    id_asistente NUMBER,
    fecha_pago TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
    monto_abonado NUMBER(10, 2) NOT NULL,
    metodo_pago VARCHAR2(30) NOT NULL,
    nro_comprobante VARCHAR2(50),
    CONSTRAINT fk_pago_plan FOREIGN KEY (id_plan_pago)
        REFERENCES PLAN_PAGO(id_plan_pago),
    CONSTRAINT fk_pago_asistente FOREIGN KEY (id_asistente)
        REFERENCES ASISTENTE(id_asistente),
    CONSTRAINT chk_monto_pago CHECK (monto_abonado > 0)
);

CREATE TABLE COMPRA (
    id_compra NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_proveedor NUMBER NOT NULL,
    id_asistente NUMBER,
    fecha_compra DATE NOT NULL,
    nro_factura VARCHAR2(50),
    total_compra NUMBER(10, 2) NOT NULL,
    estado VARCHAR2(20) DEFAULT 'Completada',
    CONSTRAINT fk_compra_proveedor FOREIGN KEY (id_proveedor)
        REFERENCES PROVEEDOR(id_proveedor),
```

```

    CONSTRAINT fk_compra_asistente FOREIGN KEY (id_asistente)
        REFERENCES ASISTENTE(id_asistente)
);

CREATE TABLE DETALLE_COMPRA (
    id_detalle_compra NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_compra NUMBER NOT NULL,
    id_suministro NUMBER NOT NULL,
    cantidad NUMBER(8,2) NOT NULL,
    precio_unitario NUMBER(10, 2) NOT NULL,
    subtotal NUMBER(10, 2) NOT NULL,
    CONSTRAINT fk_detcompra_compra FOREIGN KEY (id_compra)
        REFERENCES COMPRA(id_compra) ON DELETE CASCADE,
    CONSTRAINT fk_detcompra_suministro FOREIGN KEY (id_suministro)
        REFERENCES SUMINISTRO(id_suministro)
);

CREATE TABLE ALMACEN_INVENTARIO (
    id_inventario NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_suministro NUMBER UNIQUE NOT NULL,
    stock_actual NUMBER(8,2) DEFAULT 0 NOT NULL,
    ultima_actualizacion TIMESTAMP DEFAULT SYSTIMESTAMP,
    CONSTRAINT fk_inventario_suministro FOREIGN KEY (id_suministro)
        REFERENCES SUMINISTRO(id_suministro)
);

```

### Vista de historial clínico

```

-- =====
-- 03_vistas.sql
-- Vistas gerenciales y de consulta
-- =====

```

```

CREATE OR REPLACE VIEW VW_HISTORIAL_CLINICO AS
SELECT
    p.ci_dni AS dni_paciente,
    p.nombres || ' ' || p.apellidos AS nombre_paciente,
    d.nombres || ' ' || d.apellidos AS doctor_tratante,
    o.fecha_evaluacion,
    det.pieza_dental,
    det.diagnostico,
    t.nombre AS tratamiento_aplicado,
    det.estado AS estado_tratamiento
FROM PACIENTE p
JOIN ODONTOGRAMA o ON p.id_paciente = o.id_paciente
JOIN DOCTOR d ON o.id_doctor = d.id_doctor
JOIN DETALLE_ODONTOGRAMA det ON o.id_odontograma = det.id_odontograma
LEFT JOIN TRATAMIENTO t ON det.id_tratamiento = t.id_tratamiento;

```

## Triggers

```

-- =====
-- 04_triggers.sql
-- Disparadores de automatización e integridad operacional
-- =====

CREATE OR REPLACE TRIGGER TRG_ACTUALIZAR_STOCK_COMPRA
AFTER INSERT ON DETALLE_COMPRA
FOR EACH ROW
BEGIN
    UPDATE ALMACEN_INVENTARIO
    SET stock_actual = stock_actual + :NEW.cantidad,
        ultima_actualizacion = SYSTIMESTAMP
    WHERE id_suministro = :NEW.id_suministro;

    IF SQL%ROWCOUNT = 0 THEN

```

```

        INSERT INTO ALMACEN_INVENTARIO (id_suministro, stock_actual)
        VALUES (:NEW.id_suministro, :NEW.cantidad);
    END IF;
END;
/

CREATE OR REPLACE TRIGGER TRG_AUDITAR_PLAN_PAGO
BEFORE UPDATE OF saldo_pendiente ON PLAN_PAGO
FOR EACH ROW
BEGIN
    IF :NEW.saldo_pendiente < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Error: El saldo pendiente no puede
        ser menor a cero.');
```

```

    ELSIF :NEW.saldo_pendiente = 0 THEN
        :NEW.estado := 'Pagado';
    END IF;
END;
/
```

### Procedimiento almacenado formal

```

-- =====
-- 05_procedimientos.sql
-- Procedimientos almacenados PL/SQL
-- Proyecto: Examen Final BD3 - Farmacias
-- Motor: Oracle Database 19c
--
-- Nota para DBeaver:
-- Seleccionar desde CREATE OR REPLACE PROCEDURE hasta END SP_REGISTRAR_PAGO;
-- y ejecutar con Ctrl + Enter. No seleccionar la barra final si DBeaver
-- presenta problemas con el delimitador de script.
-- =====
```

```
CREATE OR REPLACE PROCEDURE SP_REGISTRAR_PAGO (  
    p_id_plan_pago IN NUMBER,  
    p_id_asistente IN NUMBER,  
    p_monto IN NUMBER,  
    p_metodo_pago IN VARCHAR2,  
    p_comprobante IN VARCHAR2  
)  
AS  
    v_saldo_actual PLAN_PAGO.saldo_pendiente%TYPE;  
BEGIN  
    SELECT saldo_pendiente  
    INTO v_saldo_actual  
    FROM PLAN_PAGO  
    WHERE id_plan_pago = p_id_plan_pago;  
  
    IF p_monto <= 0 THEN  
        RAISE_APPLICATION_ERROR(-20003, 'El monto del pago debe ser mayor a  
cero.');
```

```
    END IF;  
  
    IF p_monto > v_saldo_actual THEN  
        RAISE_APPLICATION_ERROR(-20002, 'El monto a pagar excede el saldo  
pendiente.');
```

```
    END IF;  
  
    INSERT INTO PAGO (  
        id_plan_pago,  
        id_asistente,  
        monto_abonado,  
        metodo_pago,  
        nro_comprobante  
    )  
    VALUES (  

```

```
        p_id_plan_pago,
        p_id_asistente,
        p_monto,
        p_metodo_pago,
        p_comprobante
    );

    UPDATE PLAN_PAGO
    SET saldo_pendiente = saldo_pendiente - p_monto
    WHERE id_plan_pago = p_id_plan_pago;

EXCEPTION

    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20004, 'El plan de pago especificado no
    existe.');
```

**END** SP\_REGISTRAR\_PAGO;

/

### Procedimiento almacenado compatible con DBeaver

```
-- =====
-- 05_procedimientos_dbeaver.sql
-- Compilación del procedimiento usando EXECUTE IMMEDIATE.
-- Uso recomendado cuando DBeaver divide incorrectamente CREATE PROCEDURE.
-- Ejecutar el bloque completo con Ctrl + Enter, SIN barra "/" final.
-- =====

BEGIN

    EXECUTE IMMEDIATE q'[
CREATE OR REPLACE PROCEDURE SP_REGISTRAR_PAGO (
    p_id_plan_pago IN NUMBER,
    p_id_asistente IN NUMBER,
    p_monto IN NUMBER,
```

```
p_metodo_pago IN VARCHAR2,
p_comprobante IN VARCHAR2
)
AS
    v_saldo_actual PLAN_PAGO.saldo_pendiente%TYPE;
BEGIN
    SELECT saldo_pendiente
    INTO v_saldo_actual
    FROM PLAN_PAGO
    WHERE id_plan_pago = p_id_plan_pago;

    IF p_monto <= 0 THEN
        RAISE_APPLICATION_ERROR(-20003, 'El monto del pago debe ser mayor a
cero.');
```

```
    END IF;

    IF p_monto > v_saldo_actual THEN
        RAISE_APPLICATION_ERROR(-20002, 'El monto a pagar excede el saldo
pendiente.');
```

```
    END IF;

    INSERT INTO PAGO (
        id_plan_pago,
        id_asistente,
        monto_abonado,
        metodo_pago,
        nro_comprobante
    )
    VALUES (
        p_id_plan_pago,
        p_id_asistente,
        p_monto,
        p_metodo_pago,
```

```
        p_comprobante
    );

    UPDATE PLAN_PAGO
    SET saldo_pendiente = saldo_pendiente - p_monto
    WHERE id_plan_pago = p_id_plan_pago;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        RAISE_APPLICATION_ERROR(-20004, 'El plan de pago especificado no
        existe.');
```

```
END SP_REGISTRAR_PAGO;
]';
END;
```

### Datos de prueba

```
-- =====
-- 06_datos_prueba.sql
-- Carga de datos de prueba para validar el esquema Farmacias_BD3
-- Proyecto: Examen Final BD3 - Farmacias
-- Motor: Oracle Database 19c
-- =====

-- Limpieza lógica de datos, respetando dependencias
DELETE FROM PAGO;
DELETE FROM PLAN_PAGO;
DELETE FROM DETALLE_ODONTOGRAMA;
DELETE FROM ODONTOGRAMA;
DELETE FROM CITA;
DELETE FROM DETALLE_COMPRA;
DELETE FROM COMPRA;
DELETE FROM ALMACEN_INVENTARIO;
```

```
DELETE FROM PACIENTE;
DELETE FROM SUMINISTRO;
DELETE FROM PROVEEDOR;
DELETE FROM DOCTOR;
DELETE FROM TRATAMIENTO;
DELETE FROM FOLDER;
DELETE FROM ASISTENTE;

COMMIT;

-- Asistentes
INSERT INTO ASISTENTE (ci_dni, nombres, apellidos, telefono, turno,
    fecha_contratacion, estado)
VALUES ('A001', 'María', 'Quispe Flores', '70000001', 'Mañana', DATE '
    2026-01-10', 1);

INSERT INTO ASISTENTE (ci_dni, nombres, apellidos, telefono, turno,
    fecha_contratacion, estado)
VALUES ('A002', 'Luis', 'Mamani Rojas', '70000002', 'Tarde', DATE '2026-02-15'
    , 1);

-- Archivadores / folders
INSERT INTO FOLDER (codigo_archivo, estante, seccion, observaciones)
VALUES ('FOL-001', 'E1', 'A', 'Historia clínica activa');

INSERT INTO FOLDER (codigo_archivo, estante, seccion, observaciones)
VALUES ('FOL-002', 'E1', 'B', 'Historia clínica activa');

-- Doctores
INSERT INTO DOCTOR (ci_dni, nombres, apellidos, especialidad, telefono, correo
    , estado)
VALUES ('D001', 'Carlos', 'Ramos Salas', 'Odontología general', '71000001', '
    carlos.ramos@demo.test', 1);
```

```
INSERT INTO DOCTOR (ci_dni, nombres, apellidos, especialidad, telefono, correo
, estado)
VALUES ('D002', 'Andrea', 'Torres Vega', 'Ortodoncia', '71000002', 'andrea.
torres@demo.test', 1);

-- Tratamientos
INSERT INTO TRATAMIENTO (nombre, descripcion, costo_referencial, estado)
VALUES ('Limpieza dental', 'Profilaxis dental básica', 120.00, 1);

INSERT INTO TRATAMIENTO (nombre, descripcion, costo_referencial, estado)
VALUES ('Curación dental', 'Restauración con resina', 180.00, 1);

INSERT INTO TRATAMIENTO (nombre, descripcion, costo_referencial, estado)
VALUES ('Ortodoncia control', 'Control mensual de ortodoncia', 250.00, 1);

-- Proveedores
INSERT INTO PROVEEDOR (nit_ruc, razon_social, nombre_contacto, telefono,
direccion, correo)
VALUES ('PRV-001', 'Dental Supply SRL', 'Roxana Paredes', '72000001', 'Av.
Central 123', 'ventas@dentalsupply.test');

INSERT INTO PROVEEDOR (nit_ruc, razon_social, nombre_contacto, telefono,
direccion, correo)
VALUES ('PRV-002', 'Insumos Odonto Bolivia', 'Javier López', '72000002', '
Calle Comercio 456', 'contacto@insumosodonto.test');

-- Suministros
INSERT INTO SUMINISTRO (codigo_barras, nombre, categoria, unidad_medida,
stock_minimo, estado)
VALUES ('SUM-001', 'Guantes descartables', 'Bioseguridad', 'Caja', 5, 1);
```

```
INSERT INTO SUMINISTRO (codigo_barras, nombre, categoria, unidad_medida,
    stock_minimo, estado)
VALUES ('SUM-002', 'Anestesia dental', 'Medicamento', 'Unidad', 10, 1);

INSERT INTO SUMINISTRO (codigo_barras, nombre, categoria, unidad_medida,
    stock_minimo, estado)
VALUES ('SUM-003', 'Resina compuesta', 'Material odontológico', 'Unidad', 3,
    1);

-- Pacientes
INSERT INTO PACIENTE (
    id_folder, ci_dni, nombres, apellidos, fecha_nacimiento, sexo,
    telefono, direccion, antecedentes_medicos
)
SELECT
    f.id_folder, 'P001', 'Daniel', 'Choque Lima', DATE '1998-05-12', 'M',
    '73000001', 'Zona Norte 100', 'Sin antecedentes relevantes'
FROM FOLDER f
WHERE f.codigo_archivo = 'FOL-001';

INSERT INTO PACIENTE (
    id_folder, ci_dni, nombres, apellidos, fecha_nacimiento, sexo,
    telefono, direccion, antecedentes_medicos
)
SELECT
    f.id_folder, 'P002', 'Lucía', 'Vargas Soto', DATE '2001-09-21', 'F',
    '73000002', 'Zona Sur 200', 'Alergia declarada a penicilina'
FROM FOLDER f
WHERE f.codigo_archivo = 'FOL-002';

-- Citas
INSERT INTO CITA (id_paciente, id_doctor, id_asistente, fecha_hora, motivo,
    estado)
```

```
SELECT p.id_paciente, d.id_doctor, a.id_asistente,
        TIMESTAMP '2026-06-20 09:00:00',
        'Evaluación inicial', 'Confirmada'
FROM PACIENTE p, DOCTOR d, ASISTENTE a
WHERE p.ci_dni = 'P001'
        AND d.ci_dni = 'D001'
        AND a.ci_dni = 'A001';

INSERT INTO CITA (id_paciente, id_doctor, id_asistente, fecha_hora, motivo,
estado)
SELECT p.id_paciente, d.id_doctor, a.id_asistente,
        TIMESTAMP '2026-06-20 10:00:00',
        'Control de ortodoncia', 'Pendiente'
FROM PACIENTE p, DOCTOR d, ASISTENTE a
WHERE p.ci_dni = 'P002'
        AND d.ci_dni = 'D002'
        AND a.ci_dni = 'A002';

-- Odontograma
INSERT INTO ODONTOGRAMA (id_paciente, id_doctor, fecha_evaluacion,
observaciones_generales)
SELECT p.id_paciente, d.id_doctor, DATE '2026-06-20', 'Evaluación inicial
registrada'
FROM PACIENTE p, DOCTOR d
WHERE p.ci_dni = 'P001'
        AND d.ci_dni = 'D001';

INSERT INTO DETALLE_ODONTOGRAMA (
        id_odontograma, id_tratamiento, pieza_dental, cara, diagnostico, estado
)
SELECT o.id_odontograma, t.id_tratamiento, '16', 'Oclusal', 'Caries inicial',
        'Por tratar'
FROM ODONTOGRAMA o
```

```
JOIN PACIENTE p ON o.id_paciente = p.id_paciente
JOIN TRATAMIENTO t ON t.nombre = 'Curación dental'
WHERE p.ci_dni = 'P001';

-- Plan de pago
INSERT INTO PLAN_PAGO (id_paciente, id_odontograma, fecha_creacion,
    costo_total, saldo_pendiente, estado)
SELECT p.id_paciente, o.id_odontograma, DATE '2026-06-20', 180.00, 180.00, '
    Vigente'
FROM PACIENTE p
JOIN ODONTOGRAMA o ON p.id_paciente = o.id_paciente
WHERE p.ci_dni = 'P001';

-- Compra y detalle de compra
INSERT INTO COMPRA (id_proveedor, id_asistente, fecha_compra, nro_factura,
    total_compra, estado)
SELECT pr.id_proveedor, a.id_asistente, DATE '2026-06-19', 'FAC-001', 450.00,
    'Completada'
FROM PROVEEDOR pr, ASISTENTE a
WHERE pr.nit_ruc = 'PRV-001'
    AND a.ci_dni = 'A001';

INSERT INTO DETALLE_COMPRA (id_compra, id_suministro, cantidad,
    precio_unitario, subtotal)
SELECT c.id_compra, s.id_suministro, 10, 25.00, 250.00
FROM COMPRA c, SUMINISTRO s
WHERE c.nro_factura = 'FAC-001'
    AND s.codigo_barras = 'SUM-001';

INSERT INTO DETALLE_COMPRA (id_compra, id_suministro, cantidad,
    precio_unitario, subtotal)
SELECT c.id_compra, s.id_suministro, 5, 40.00, 200.00
FROM COMPRA c, SUMINISTRO s
```

```
WHERE c.nro_factura = 'FAC-001'
      AND s.codigo_barras = 'SUM-003';

COMMIT;
```

### Ejecutor general

```
-- =====
-- 99_ejecutar_todo.sql
-- Ejecutor principal para DBeaver / Oracle 19c
-- Ejecutar desde el usuario/schema del proyecto
-- =====

@@01_limpieza_esquema.sql
@@02_tablas.sql
@@03_vistas.sql
@@04_triggers.sql
@@05_procedimientos.sql
```

## Apéndice B

### Anexo: scripts de validación

#### Validación de estructura

```
-- =====
-- 01_validacion_estructura.sql
-- Validación de objetos creados en Oracle Database 19c
-- Proyecto: Examen Final BD3 - Farmacias
-- =====

-- 1. Tablas creadas
SELECT table_name
FROM user_tables
ORDER BY table_name;

-- 2. Conteo general de objetos por tipo
SELECT object_type, COUNT(*) AS cantidad
FROM user_objects
WHERE object_name IN (
    'ASISTENTE',
    'FOLDER',
    'TRATAMIENTO',
    'DOCTOR',
    'PROVEEDOR',
    'SUMINISTRO',
    'PACIENTE',
    'CITA',
    'ODONTOGRAMA',
    'DETALLE_ODONTOGRAMA',
    'PLAN_PAGO',
    'PAGO',
    'COMPRA',
    'DETALLE_COMPRA',
```

```
        'ALMACEN_INVENTARIO',  
        'VW_HISTORIAL_CLINICO',  
        'TRG_ACTUALIZAR_STOCK_COMPRA',  
        'TRG_AUDITAR_PLAN_PAGO',  
        'SP_REGISTRAR_PAGO'  
    )
```

```
GROUP BY object_type
```

```
ORDER BY object_type;
```

```
-- 3. Restricciones por tabla
```

```
SELECT
```

```
    table_name,  
    constraint_name,  
    constraint_type,  
    status
```

```
FROM user_constraints
```

```
WHERE table_name IN (
```

```
    'ASISTENTE',  
    'FOLDER',  
    'TRATAMIENTO',  
    'DOCTOR',  
    'PROVEEDOR',  
    'SUMINISTRO',  
    'PACIENTE',  
    'CITA',  
    'ODONTOGRAMA',  
    'DETALLE_ODONTOGRAMA',  
    'PLAN_PAGO',  
    'PAGO',  
    'COMPRA',  
    'DETALLE_COMPRA',  
    'ALMACEN_INVENTARIO'
```

```
)
```

```
ORDER BY table_name, constraint_type, constraint_name;
```

```
-- 4. Claves foráneas
```

```
SELECT
```

```
    uc.table_name,  
    uc.constraint_name,  
    ucc.column_name,  
    ruc.table_name AS tabla_referenciada,  
    rucc.column_name AS columna_referenciada
```

```
FROM user_constraints uc
```

```
JOIN user_cons_columns ucc
```

```
    ON uc.constraint_name = ucc.constraint_name
```

```
JOIN user_constraints ruc
```

```
    ON uc.r_constraint_name = ruc.constraint_name
```

```
JOIN user_cons_columns rucc
```

```
    ON ruc.constraint_name = rucc.constraint_name
```

```
WHERE uc.constraint_type = 'R'
```

```
ORDER BY uc.table_name, uc.constraint_name, ucc.position;
```

```
-- 5. Estado de vista, triggers y procedimiento
```

```
SELECT
```

```
    object_name,  
    object_type,  
    status
```

```
FROM user_objects
```

```
WHERE object_name IN (
```

```
    'VW_HISTORIAL_CLINICO',  
    'TRG_ACTUALIZAR_STOCK_COMPRA',  
    'TRG_AUDITAR_PLAN_PAGO',  
    'SP_REGISTRAR_PAGO'
```

```
)
```

```
ORDER BY object_type, object_name;
```

**Validación de datos de prueba**

```
-- =====
-- 02_validacion_datos_prueba.sql
-- Validaciones posteriores a la carga de datos de prueba
-- Proyecto: Examen Final BD3 - Farmacias
-- Motor: Oracle Database 19c
-- =====

-- 1. Cantidad de registros por tabla principal
SELECT nombre_tabla, cantidad
FROM (
    SELECT 'ASISTENTE' AS nombre_tabla, COUNT(*) AS cantidad FROM ASISTENTE
    UNION ALL SELECT 'FOLDER', COUNT(*) FROM FOLDER
    UNION ALL SELECT 'DOCTOR', COUNT(*) FROM DOCTOR
    UNION ALL SELECT 'TRATAMIENTO', COUNT(*) FROM TRATAMIENTO
    UNION ALL SELECT 'PROVEEDOR', COUNT(*) FROM PROVEEDOR
    UNION ALL SELECT 'SUMINISTRO', COUNT(*) FROM SUMINISTRO
    UNION ALL SELECT 'PACIENTE', COUNT(*) FROM PACIENTE
    UNION ALL SELECT 'CITA', COUNT(*) FROM CITA
    UNION ALL SELECT 'ODONTOGRAMA', COUNT(*) FROM ODONTOGRAMA
    UNION ALL SELECT 'DETALLE_ODONTOGRAMA', COUNT(*) FROM DETALLE_ODONTOGRAMA
    UNION ALL SELECT 'PLAN_PAGO', COUNT(*) FROM PLAN_PAGO
    UNION ALL SELECT 'COMPRA', COUNT(*) FROM COMPRA
    UNION ALL SELECT 'DETALLE_COMPRA', COUNT(*) FROM DETALLE_COMPRA
    UNION ALL SELECT 'ALMACEN_INVENTARIO', COUNT(*) FROM ALMACEN_INVENTARIO
)
ORDER BY 1;

-- 2. Validación del trigger de inventario
SELECT
    s.codigo_barras,
    s.nombre,
```

```
        ai.stock_actual,
        ai.ultima_actualizacion
FROM SUMINISTRO s
JOIN ALMACEN_INVENTARIO ai
      ON s.id_suministro = ai.id_suministro
ORDER BY s.codigo_barras;

-- 3. Validación de la vista de historial clínico
SELECT
    dni_paciente,
    nombre_paciente,
    doctor_tratante,
    fecha_evaluacion,
    pieza_dental,
    diagnostico,
    tratamiento_aplicado,
    estado_tratamiento
FROM VW_HISTORIAL_CLINICO
ORDER BY fecha_evaluacion, dni_paciente;

-- 4. Validación de planes de pago
SELECT
    pp.id_plan_pago,
    p.ci_dni,
    p.nombres || ' ' || p.apellidos AS paciente,
    pp.costo_total,
    pp.saldo_pendiente,
    pp.estado
FROM PLAN_PAGO pp
JOIN PACIENTE p
      ON pp.id_paciente = p.id_paciente
ORDER BY pp.id_plan_pago;
```

**Prueba del procedimiento de pago**

```
-- =====
-- 03_prueba_procedimiento_pago.sql
-- Prueba funcional re-ejecutable del procedimiento SP_REGISTRAR_PAGO
-- Proyecto: Examen Final BD3 - Farmacias
-- Motor: Oracle Database 19c
-- =====

-- 1. Reiniciar la prueba para evitar doble descuento si se ejecuta más de una
    vez
DECLARE
    v_id_plan_pago PLAN_PAGO.id_plan_pago%TYPE;
BEGIN
    SELECT pp.id_plan_pago
    INTO v_id_plan_pago
    FROM PLAN_PAGO pp
    JOIN PACIENTE p
    ON pp.id_paciente = p.id_paciente
    WHERE p.ci_dni = 'P001';

    DELETE FROM PAGO
    WHERE id_plan_pago = v_id_plan_pago
    AND nro_comprobante = 'REC-001';

    UPDATE PLAN_PAGO
    SET saldo_pendiente = costo_total,
        estado = 'Vigente'
    WHERE id_plan_pago = v_id_plan_pago;

    COMMIT;
END;
```

```
-- 2. Saldo antes del pago
```

```
SELECT
```

```
    pp.id_plan_pago,  
    p.ci_dni,  
    p.nombres || ' ' || p.apellidos AS paciente,  
    pp.costo_total,  
    pp.saldo_pendiente,  
    pp.estado
```

```
FROM PLAN_PAGO pp
```

```
JOIN PACIENTE p
```

```
    ON pp.id_paciente = p.id_paciente
```

```
WHERE p.ci_dni = 'P001';
```

```
-- 3. Ejecutar pago parcial
```

```
DECLARE
```

```
    v_id_plan_pago PLAN_PAGO.id_plan_pago%TYPE;
```

```
    v_id_asistente ASISTENTE.id_asistente%TYPE;
```

```
BEGIN
```

```
    SELECT pp.id_plan_pago
```

```
    INTO v_id_plan_pago
```

```
    FROM PLAN_PAGO pp
```

```
    JOIN PACIENTE p
```

```
        ON pp.id_paciente = p.id_paciente
```

```
    WHERE p.ci_dni = 'P001';
```

```
    SELECT id_asistente
```

```
    INTO v_id_asistente
```

```
    FROM ASISTENTE
```

```
    WHERE ci_dni = 'A001';
```

```
SP_REGISTRAR_PAGO(
```

```
    p_id_plan_pago => v_id_plan_pago,
```

```
    p_id_asistente => v_id_asistente,
```

```
        p_monto => 80,
        p_metodo_pago => 'Efectivo',
        p_comprobante => 'REC-001'
    );

    COMMIT;
END;

-- 4. Pagos registrados
SELECT
    pg.id_pago,
    pg.id_plan_pago,
    pg.id_asistente,
    pg.monto_abonado,
    pg.metodo_pago,
    pg.nro_comprobante,
    pg.fecha_pago
FROM PAGO pg
WHERE pg.nro_comprobante = 'REC-001'
ORDER BY pg.id_pago;

-- 5. Saldo después del pago
SELECT
    pp.id_plan_pago,
    p.ci_dni,
    p.nombres || ' ' || p.apellidos AS paciente,
    pp.costo_total,
    pp.saldo_pendiente,
    pp.estado
FROM PLAN_PAGO pp
JOIN PACIENTE p
    ON pp.id_paciente = p.id_paciente
WHERE p.ci_dni = 'P001';
```

---

## Diccionario técnico extraído desde Oracle

```
-- =====
-- 04_diccionario_tecnico_oracle.sql
-- Extracción técnica del diccionario desde Oracle Database 19c
-- Proyecto: Examen Final BD3 - Farmacias
-- =====

-- 1. Columnas, tipos de datos, nulabilidad y orden físico
SELECT
    utc.table_name,
    utc.column_id,
    utc.column_name,
    utc.data_type,
    CASE
        WHEN utc.data_type IN ('VARCHAR2', 'CHAR') THEN TO_CHAR(utc.
data_length)
        WHEN utc.data_type = 'NUMBER' AND utc.data_precision IS NOT NULL THEN
utc.data_precision || ',' || utc.data_scale
        ELSE NULL
    END AS longitud_precision,
    utc.nullable,
    utc.data_default
FROM user_tab_columns utc
WHERE utc.table_name IN (
    'ASISTENTE',
    'FOLDER',
    'TRATAMIENTO',
    'DOCTOR',
    'PROVEEDOR',
    'SUMINISTRO',
    'PACIENTE',
```

```
'CITA',
'ODONTOGRAMA',
'DETALLE_ODONTOGRAMA',
'PLAN_PAGO',
'PAGO',
'COMPRA',
'DETALLE_COMPRA',
'ALMACEN_INVENTARIO'
)
ORDER BY utc.table_name, utc.column_id;

-- 2. Restricciones primarias, únicas, foráneas y check
SELECT
    uc.table_name,
    uc.constraint_name,
    CASE uc.constraint_type
        WHEN 'P' THEN 'PRIMARY KEY'
        WHEN 'R' THEN 'FOREIGN KEY'
        WHEN 'U' THEN 'UNIQUE'
        WHEN 'C' THEN 'CHECK / NOT NULL'
        ELSE uc.constraint_type
    END AS tipo_restriccion,
    uc.status,
    uc.delete_rule,
    uc.search_condition_vc AS condicion_check
FROM user_constraints uc
WHERE uc.table_name IN (
    'ASISTENTE',
    'FOLDER',
    'TRATAMIENTO',
    'DOCTOR',
    'PROVEEDOR',
    'SUMINISTRO',
```

```
'PACIENTE',
'CITA',
'ODONTOGRAMA',
'DETALLE_ODONTOGRAMA',
'PLAN_PAGO',
'PAGO',
'COMPRA',
'DETALLE_COMPRA',
'ALMACEN_INVENTARIO'
)
ORDER BY uc.table_name, uc.constraint_type, uc.constraint_name;

-- 3. Columnas que participan en restricciones
SELECT
    ucc.table_name,
    ucc.constraint_name,
    ucc.column_name,
    ucc.position
FROM user_cons_columns ucc
WHERE ucc.table_name IN (
    'ASISTENTE',
    'FOLDER',
    'TRATAMIENTO',
    'DOCTOR',
    'PROVEEDOR',
    'SUMINISTRO',
    'PACIENTE',
    'CITA',
    'ODONTOGRAMA',
    'DETALLE_ODONTOGRAMA',
    'PLAN_PAGO',
    'PAGO',
    'COMPRA',
```

```
'DETALLE_COMPRA',
'ALMACEN_INVENTARIO'
)
ORDER BY ucc.table_name, ucc.constraint_name, ucc.position;

-- 4. Relaciones de claves foráneas
SELECT
    child.table_name AS tabla_hija,
    child_cols.column_name AS columna_hija,
    child.constraint_name AS fk_name,
    parent.table_name AS tabla_padre,
    parent_cols.column_name AS columna_padre,
    child.delete_rule
FROM user_constraints child
JOIN user_cons_columns child_cols
    ON child.constraint_name = child_cols.constraint_name
JOIN user_constraints parent
    ON child.r_constraint_name = parent.constraint_name
JOIN user_cons_columns parent_cols
    ON parent.constraint_name = parent_cols.constraint_name
    AND child_cols.position = parent_cols.position
WHERE child.constraint_type = 'R'
ORDER BY child.table_name, child.constraint_name, child_cols.position;

-- 5. Vista, triggers y procedimiento documentados
SELECT
    object_name,
    object_type,
    status
FROM user_objects
WHERE object_name IN (
    'VW_HISTORIAL_CLINICO',
    'TRG_ACTUALIZAR_STOCK_COMPRA',
```

```
'TRG_AUDITAR_PLAN_PAGO',  
'SP_REGISTRAR_PAGO'  
)  
ORDER BY object_type, object_name;
```

## Apéndice C

### Anexo: diccionario de datos completo

El diccionario de datos completo se incluye como anexo para mantener trazabilidad entre la documentación técnica y el esquema implementado. A diferencia del archivo fuente en formato Markdown, en este anexo las tablas fueron convertidas a formato LaTeX para que se visualicen correctamente dentro del PDF final.

### Diccionario de datos

#### *Descripción general*

El presente diccionario de datos documenta las tablas, columnas, tipos de datos, claves, restricciones y propósito funcional del esquema implementado en Oracle Database 19c. El modelo físico corresponde a una base de datos orientada a la administración de procesos de registro, atención clínica odontológica, inventario y pagos.

Aunque el schema del proyecto se denomina FARMACIAS\_BD3 por convención del repositorio y del entorno académico, las entidades implementadas corresponden al dominio del sistema odontológico analizado a partir de la tesis base.

#### ***Tabla: ASISTENTE***

**Propósito:** almacena los datos del personal asistente que participa en la gestión de citas, compras y pagos.

| Campo              | Tipo Oracle  | Nulo | Clave | Descripción                                       |
|--------------------|--------------|------|-------|---------------------------------------------------|
| id_asistente       | NUMBER       | No   | PK    | Identificador único generado automáticamente.     |
| ci_dni             | VARCHAR2(20) | No   | UK    | Documento de identidad del asistente.             |
| nombres            | VARCHAR2(80) | No   |       | Nombres del asistente.                            |
| apellidos          | VARCHAR2(80) | No   |       | Apellidos del asistente.                          |
| telefono           | VARCHAR2(15) | Sí   |       | Número telefónico de contacto.                    |
| turno              | VARCHAR2(20) | Sí   |       | Turno asignado al asistente.                      |
| fecha_contratacion | DATE         | Sí   |       | Fecha de contratación.                            |
| estado             | NUMBER(1)    | No   |       | Estado lógico del registro. Valor por defecto: 1. |

#### ***Tabla: FOLDER***

**Propósito:** representa el archivador o carpeta física/digital donde se ubica la documentación clínica del paciente.

| Campo          | Tipo Oracle   | Nulo | Clave | Descripción                                   |
|----------------|---------------|------|-------|-----------------------------------------------|
| id_folder      | NUMBER        | No   | PK    | Identificador único generado automáticamente. |
| codigo_archivo | VARCHAR2(20)  | No   | UK    | Código único del archivador o carpeta.        |
| estante        | VARCHAR2(20)  | Sí   |       | Estante donde se ubica el archivo.            |
| seccion        | VARCHAR2(20)  | Sí   |       | Sección dentro del estante.                   |
| observaciones  | VARCHAR2(255) | Sí   |       | Notas adicionales sobre el archivo.           |

**Tabla: TRATAMIENTO**

**Propósito:** catálogo de tratamientos odontológicos disponibles.

| Campo             | Tipo Oracle   | Nulo | Clave | Descripción                                          |
|-------------------|---------------|------|-------|------------------------------------------------------|
| id_tratamiento    | NUMBER        | No   | PK    | Identificador único generado automáticamente.        |
| nombre            | VARCHAR2(100) | No   |       | Nombre del tratamiento.                              |
| descripcion       | VARCHAR2(255) | Sí   |       | Descripción breve del tratamiento.                   |
| costo_referencial | NUMBER(10,2)  | No   |       | Costo referencial del tratamiento.                   |
| estado            | NUMBER(1)     | No   |       | Estado lógico del tratamiento. Valor por defecto: 1. |

**Tabla: DOCTOR**

**Propósito:** almacena los datos de los doctores u odontólogos que atienden pacientes y registran odontogramas.

| Campo        | Tipo Oracle   | Nulo | Clave | Descripción                                       |
|--------------|---------------|------|-------|---------------------------------------------------|
| id_doctor    | NUMBER        | No   | PK    | Identificador único generado automáticamente.     |
| ci_dni       | VARCHAR2(20)  | No   | UK    | Documento de identidad del doctor.                |
| nombres      | VARCHAR2(80)  | No   |       | Nombres del doctor.                               |
| apellidos    | VARCHAR2(80)  | No   |       | Apellidos del doctor.                             |
| especialidad | VARCHAR2(100) | Sí   |       | Especialidad odontológica.                        |
| telefono     | VARCHAR2(15)  | Sí   |       | Teléfono de contacto.                             |
| correo       | VARCHAR2(100) | Sí   | UK    | Correo electrónico único.                         |
| estado       | NUMBER(1)     | No   |       | Estado lógico del registro. Valor por defecto: 1. |

**Tabla: PROVEEDOR**

**Propósito:** almacena proveedores de suministros, materiales e insumos.

| Campo           | Tipo Oracle   | Nulo | Clave | Descripción                                        |
|-----------------|---------------|------|-------|----------------------------------------------------|
| id_proveedor    | NUMBER        | No   | PK    | Identificador único generado automáticamente.      |
| nit_ruc         | VARCHAR2(20)  | No   | UK    | Número de identificación tributaria del proveedor. |
| razon_social    | VARCHAR2(100) | No   |       | Razón social del proveedor.                        |
| nombre_contacto | VARCHAR2(80)  | Sí   |       | Nombre de la persona de contacto.                  |
| telefono        | VARCHAR2(15)  | Sí   |       | Teléfono de contacto.                              |
| direccion       | VARCHAR2(255) | Sí   |       | Dirección del proveedor.                           |
| correo          | VARCHAR2(100) | Sí   |       | Correo electrónico del proveedor.                  |

**Tabla: SUMINISTRO**

**Propósito:** catálogo de suministros utilizados en la atención odontológica y administración del inventario.

| Campo         | Tipo Oracle   | Nulo | Clave | Descripción                                         |
|---------------|---------------|------|-------|-----------------------------------------------------|
| id_suministro | NUMBER        | No   | PK    | Identificador único generado automáticamente.       |
| codigo_barras | VARCHAR2(50)  | Sí   | UK    | Código de barras o código interno del suministro.   |
| nombre        | VARCHAR2(100) | No   |       | Nombre del suministro.                              |
| categoria     | VARCHAR2(50)  | Sí   |       | Categoría del suministro.                           |
| unidad_medida | VARCHAR2(20)  | Sí   |       | Unidad de medida usada para controlar stock.        |
| stock_minimo  | NUMBER(5)     | No   |       | Cantidad mínima recomendada en inventario.          |
| estado        | NUMBER(1)     | No   |       | Estado lógico del suministro. Valor por defecto: 1. |

**Tabla: PACIENTE**

**Propósito:** registra los datos personales y antecedentes clínicos básicos de los pacientes.

| Campo                | Tipo Oracle   | Nulo | Clave | Descripción                                                 |
|----------------------|---------------|------|-------|-------------------------------------------------------------|
| id_paciente          | NUMBER        | No   | PK    | Identificador único generado automáticamente.               |
| id_folder            | NUMBER        | Sí   | FK    | Referencia al archivador físico o digital del paciente.     |
| ci_dni               | VARCHAR2(20)  | No   | UK    | Documento de identidad del paciente.                        |
| nombres              | VARCHAR2(80)  | No   |       | Nombres del paciente.                                       |
| apellidos            | VARCHAR2(80)  | No   |       | Apellidos del paciente.                                     |
| fecha_nacimiento     | DATE          | No   |       | Fecha de nacimiento.                                        |
| sexo                 | CHAR(1)       | Sí   |       | Sexo registrado del paciente.                               |
| telefono             | VARCHAR2(15)  | Sí   |       | Teléfono de contacto.                                       |
| direccion            | VARCHAR2(255) | Sí   |       | Dirección del paciente.                                     |
| antecedentes_medicos | VARCHAR2(500) | Sí   |       | Antecedentes médicos declarados.                            |
| fecha_registro       | DATE          | Sí   |       | Fecha de registro del paciente. Valor por defecto: SYSDATE. |

**Relaciones:**

PACIENTE.id\_folder referencia FOLDER.id\_folder con ON DELETE SET NULL.

**Tabla: CITA**

**Propósito:** registra la programación de atenciones odontológicas.

| Campo          | Tipo Oracle   | Nulo | Clave | Descripción                                                 |
|----------------|---------------|------|-------|-------------------------------------------------------------|
| id_cita        | NUMBER        | No   | PK    | Identificador único generado automáticamente.               |
| id_paciente    | NUMBER        | No   | FK    | Paciente citado.                                            |
| id_doctor      | NUMBER        | No   | FK    | Doctor asignado.                                            |
| id_asistente   | NUMBER        | Sí   | FK    | Asistente relacionado con la cita.                          |
| fecha_hora     | TIMESTAMP     | No   |       | Fecha y hora programada.                                    |
| motivo         | VARCHAR2(255) | No   |       | Motivo de la cita.                                          |
| estado         | VARCHAR2(20)  | No   | CHECK | Estado de la cita.                                          |
| fecha_creacion | DATE          | Sí   |       | Fecha de creación del registro. Valor por defecto: SYSDATE. |

**Valores permitidos para estado:** Pendiente, Confirmada, Atendida, Cancelada, Reprogramada.

**Relaciones:**

CITA.id\_paciente referencia PACIENTE.id\_paciente.  
 CITA.id\_doctor referencia DOCTOR.id\_doctor.  
 CITA.id\_asistente referencia ASISTENTE.id\_asistente.

### **Tabla: ODONTOGRAMA**

**Propósito:** representa la evaluación odontológica general de un paciente.

| Campo                   | Tipo Oracle   | Nulo | Clave | Descripción                                       |
|-------------------------|---------------|------|-------|---------------------------------------------------|
| id_odontograma          | NUMBER        | No   | PK    | Identificador único generado automáticamente.     |
| id_paciente             | NUMBER        | No   | FK    | Paciente evaluado.                                |
| id_doctor               | NUMBER        | No   | FK    | Doctor que registra la evaluación.                |
| fecha_evaluacion        | DATE          | No   |       | Fecha de evaluación. Valor por defecto: SYS-DATE. |
| observaciones_generales | VARCHAR2(500) | Sí   |       | Observaciones clínicas generales.                 |

**Relaciones:**

ODONTOGRAMA.id\_paciente referencia PACIENTE.id\_paciente.  
 ODONTOGRAMA.id\_doctor referencia DOCTOR.id\_doctor.

### **Tabla: DETALLE\_ODONTOGRAMA**

**Propósito:** registra el detalle clínico por pieza dental, cara, diagnóstico y tratamiento asociado.

| Campo          | Tipo Oracle   | Nulo | Clave | Descripción                                                       |
|----------------|---------------|------|-------|-------------------------------------------------------------------|
| id_detalle     | NUMBER        | No   | PK    | Identificador único generado automáticamente.                     |
| id_odontograma | NUMBER        | No   | FK    | Odontograma al que pertenece el detalle.                          |
| id_tratamiento | NUMBER        | Sí   | FK    | Tratamiento asociado, si corresponde.                             |
| pieza_dental   | VARCHAR2(10)  | No   |       | Código o número de pieza dental.                                  |
| cara           | VARCHAR2(20)  | No   |       | Cara dental evaluada.                                             |
| diagnostico    | VARCHAR2(100) | No   |       | Diagnóstico registrado.                                           |
| estado         | VARCHAR2(20)  | Sí   |       | Estado del tratamiento o atención. Valor por defecto: Por tratar. |

**Relaciones:**

DETALLE\_ODONTOGRAMA.id\_odontograma referencia ODONTOGRAMA.id\_odontograma con ON DELETE CASCADE.  
 DETALLE\_ODONTOGRAMA.id\_tratamiento referencia TRATAMIENTO.id\_tratamiento.

**Tabla: PLAN\_PAGO**

**Propósito:** registra el costo total y saldo pendiente de los tratamientos asociados a un odontograma.

| Campo           | Tipo Oracle  | Nulo | Clave | Descripción                                             |
|-----------------|--------------|------|-------|---------------------------------------------------------|
| id_plan_pago    | NUMBER       | No   | PK    | Identificador único generado automáticamente.           |
| id_paciente     | NUMBER       | No   | FK    | Paciente asociado al plan.                              |
| id_odontograma  | NUMBER       | No   | FK    | Odontograma relacionado.                                |
| fecha_creacion  | DATE         | Sí   |       | Fecha de creación del plan. Valor por defecto: SYSDATE. |
| costo_total     | NUMBER(10,2) | No   |       | Costo total del plan.                                   |
| saldo_pendiente | NUMBER(10,2) | No   |       | Saldo pendiente de pago.                                |
| estado          | VARCHAR2(20) | Sí   |       | Estado del plan. Valor por defecto: Vigente.            |

**Relaciones:**

PLAN\_PAGO.id\_paciente referencia PACIENTE.id\_paciente.

PLAN\_PAGO.id\_odontograma referencia ODONTOGRAMA.id\_odontograma.

**Regla automatizada:**

El trigger TRG\_AUDITAR\_PLAN\_PAGO impide que el saldo pendiente sea negativo y cambia el estado a Pagado cuando el saldo llega a cero.

**Tabla: PAGO**

**Propósito:** registra los pagos efectuados por los pacientes sobre un plan de pago.

| Campo           | Tipo Oracle  | Nulo | Clave | Descripción                                              |
|-----------------|--------------|------|-------|----------------------------------------------------------|
| id_pago         | NUMBER       | No   | PK    | Identificador único generado automáticamente.            |
| id_plan_pago    | NUMBER       | No   | FK    | Plan de pago asociado.                                   |
| id_asistente    | NUMBER       | Sí   | FK    | Asistente que registra el pago.                          |
| fecha_pago      | TIMESTAMP    | No   |       | Fecha y hora del pago. Valor por defecto: SYS-TIMESTAMP. |
| monto_abonado   | NUMBER(10,2) | No   | CHECK | Monto abonado. Debe ser mayor a cero.                    |
| metodo_pago     | VARCHAR2(30) | No   |       | Método de pago utilizado.                                |
| nro_comprobante | VARCHAR2(50) | Sí   |       | Número de recibo o comprobante.                          |

**Relaciones:**

PAGO.id\_plan\_pago referencia PLAN\_PAGO.id\_plan\_pago.

PAGO.id\_asistente referencia ASISTENTE.id\_asistente.

**Regla:**

CHK\_MONTO\_PAGO exige que monto\_abonado > 0.

**Tabla: COMPRA**

**Propósito:** registra compras de suministros realizadas a proveedores.

| Campo        | Tipo Oracle  | Nulo | Clave | Descripción                                         |
|--------------|--------------|------|-------|-----------------------------------------------------|
| id_compra    | NUMBER       | No   | PK    | Identificador único generado automáticamente.       |
| id_proveedor | NUMBER       | No   | FK    | Proveedor de la compra.                             |
| id_asistente | NUMBER       | Sí   | FK    | Asistente que registra la compra.                   |
| fecha_compra | DATE         | No   |       | Fecha de la compra.                                 |
| nro_factura  | VARCHAR2(50) | Sí   |       | Número de factura.                                  |
| total_compra | NUMBER(10,2) | No   |       | Total monetario de la compra.                       |
| estado       | VARCHAR2(20) | Sí   |       | Estado de la compra. Valor por defecto: Completada. |

**Relaciones:**

COMPRA.id\_proveedor referencia PROVEEDOR.id\_proveedor.

COMPRA.id\_asistente referencia ASISTENTE.id\_asistente.

**Tabla: DETALLE\_COMPRA**

**Propósito:** registra los suministros incluidos en cada compra.

| Campo             | Tipo Oracle  | Nulo | Clave | Descripción                                   |
|-------------------|--------------|------|-------|-----------------------------------------------|
| id_detalle_compra | NUMBER       | No   | PK    | Identificador único generado automáticamente. |
| id_compra         | NUMBER       | No   | FK    | Compra a la que pertenece el detalle.         |
| id_suministro     | NUMBER       | No   | FK    | Suministro comprado.                          |
| cantidad          | NUMBER(8,2)  | No   |       | Cantidad comprada.                            |
| precio_unitario   | NUMBER(10,2) | No   |       | Precio unitario del suministro.               |
| subtotal          | NUMBER(10,2) | No   |       | Subtotal del detalle.                         |

**Relaciones:**

DETALLE\_COMPRA.id\_compra referencia COMPRA.id\_compra con ON DELETE CASCADE.

DETALLE\_COMPRA.id\_suministro referencia SUMINISTRO.id\_suministro.

**Regla automatizada:**

El trigger TRG\_ACTUALIZAR\_STOCK\_COMPRA actualiza o crea el registro correspondiente en ALMACEN\_INVENTARIO después de insertar un detalle de compra.

**Tabla: ALMACEN\_INVENTARIO**

**Propósito:** controla el stock actual por suministro.

| Campo                | Tipo Oracle | Nulo | Clave   | Descripción                                                            |
|----------------------|-------------|------|---------|------------------------------------------------------------------------|
| id_inventario        | NUMBER      | No   | PK      | Identificador único generado automáticamente.                          |
| id_suministro        | NUMBER      | No   | FK / UK | Suministro inventariado. Debe ser único en inventario.                 |
| stock_actual         | NUMBER(8,2) | No   |         | Stock disponible actual.                                               |
| ultima_actualizacion | TIMESTAMP   | Sí   |         | Fecha y hora de última actualización. Valor por defecto: SYSTIMESTAMP. |

**Relaciones:**

ALMACEN\_INVENTARIO.id\_suministro referencia SUMINISTRO.id\_suministro.

**Vista: VW\_HISTORIAL\_CLINICO**

**Propósito:** integra información de pacientes, doctores, odontogramas, detalles clínicos y tratamientos para consultar el historial clínico odontológico.

| Columna              | Descripción                                           |
|----------------------|-------------------------------------------------------|
| dni_paciente         | Documento de identidad del paciente.                  |
| nombre_paciente      | Nombre completo del paciente.                         |
| doctor_tratante      | Nombre completo del doctor tratante.                  |
| fecha_evaluacion     | Fecha de evaluación del odontograma.                  |
| pieza_dental         | Pieza dental evaluada.                                |
| diagnostico          | Diagnóstico registrado.                               |
| tratamiento_aplicado | Tratamiento asociado al diagnóstico.                  |
| estado_tratamiento   | Estado del tratamiento en el detalle del odontograma. |

**Triggers**

| Trigger                      | Tabla          | Evento       | Propósito                                                                |
|------------------------------|----------------|--------------|--------------------------------------------------------------------------|
| TRG_ACTUALI-ZAR_STOCK_COMPRA | DETALLE_COMPRA | AFTER INSERT | Actualiza o crea el stock del suministro comprado en ALMACEN_INVENTARIO. |

| Trigger               | Tabla     | Evento                           | Propósito                                                                                 |
|-----------------------|-----------|----------------------------------|-------------------------------------------------------------------------------------------|
| TRG_AUDITAR_PLAN_PAGO | PLAN_PAGO | BEFORE UPDATE OF saldo_pendiente | Evita saldos negativos y cambia el estado del plan a Pagado cuando el saldo llega a cero. |

### *Procedimientos almacenados*

| Procedimiento     | Propósito                                                                                                                                                    |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SP_REGISTRAR_PAGO | Registra un pago, valida que el monto sea positivo, valida que no exceda el saldo pendiente, inserta el registro en PAGO y actualiza el saldo del PLAN_PAGO. |

### *Resumen de integridad referencial*

| Tabla hija          | Columna FK     | Tabla padre | Columna PK     | Acción             |
|---------------------|----------------|-------------|----------------|--------------------|
| PACIENTE            | id_folder      | FOLDER      | id_folder      | ON DELETE SET NULL |
| CITA                | id_paciente    | PACIENTE    | id_paciente    | Restictiva         |
| CITA                | id_doctor      | DOCTOR      | id_doctor      | Restictiva         |
| CITA                | id_asistente   | ASISTENTE   | id_asistente   | Restictiva         |
| ODONTOGRAMA         | id_paciente    | PACIENTE    | id_paciente    | Restictiva         |
| ODONTOGRAMA         | id_doctor      | DOCTOR      | id_doctor      | Restictiva         |
| DETALLE_ODONTOGRAMA | id_odontograma | ODONTOGRAMA | id_odontograma | ON DELETE CASCADE  |
| DETALLE_ODONTOGRAMA | id_tratamiento | TRATAMIENTO | id_tratamiento | Restictiva         |
| PLAN_PAGO           | id_paciente    | PACIENTE    | id_paciente    | Restictiva         |
| PLAN_PAGO           | id_odontograma | ODONTOGRAMA | id_odontograma | Restictiva         |

| Tabla hija         | Columna FK    | Tabla padre | Columna PK    | Acción            |
|--------------------|---------------|-------------|---------------|-------------------|
| PAGO               | id_plan_pago  | PLAN_PAGO   | id_plan_pago  | Restictiva        |
| PAGO               | id_asistente  | ASISTENTE   | id_asistente  | Restictiva        |
| COMPRA             | id_proveedor  | PROVEEDOR   | id_proveedor  | Restictiva        |
| COMPRA             | id_asistente  | ASISTENTE   | id_asistente  | Restictiva        |
| DETALLE_COMPRA     | id_compra     | COMPRA      | id_compra     | ON DELETE CASCADE |
| DETALLE_COMPRA     | id_suministro | SUMINISTRO  | id_suministro | Restictiva        |
| ALMACEN_INVENTARIO | id_suministro | SUMINISTRO  | id_suministro | Restictiva        |